# PostgreSQL

Содержание:


- Основные понятия
- SQL язык
  - Группы операторов
- Операции с таблицами
  - Создание таблиц
  - Удаление таблиц
  - Вставка данных в таблицу
  - Выборка данных
  - Выборка определенных строк
  - Обновление данных
  - Ограничение количества строк вывода
- Типы данных
  - Числовые
  - Символьные
  - Дата и время
  - Логические типы
  - JSON
  - Массивы
  - Сетевые
  - UUID
  - Бинарные
  - Геометрические
  - Каскадное удаление
- Подсчет строк
- Группировка результата
- Агрегатные функции
- Работа с датами
- Массивы
- JSON и JSONB
- Constraint
- Выбор схемы и удаление таблиц
  - Про схемы
- Изменение таблиц

- Представления
- Аномалии и транзакции
  - ACID
    - Atomicity
    - Consistency
    - Isolation
    - Durability
  - Аномалии
    - Dirty read
    - Non-repeatable read
    - Phantom read
    - Lost update
    - Serialization anomaly
  - Уровни изоляции
    - Read Uncommitted
    - Read Committed
    - Repeatable Read
    - Serializable
- Дополнительно про запросы
  - LIKE и NOT LIKE
- Условная конструкция
- Join'ы
  - INNER JOIN / JOIN
  - LEFT JOIN
  - RIGHT JOIN
  - OUTER JOIN
- Множества
  - UNION
  - INTERSECT
  - EXCEPT
  - Замена запросов с UNION, INTERSECT, EXCEPT
- Оконные функции
  - Порядок выполнения
  - Синтаксис оконных функций
  - Классы оконных функций
- DISTINCT
- Подзапросы
  - Как работает `SELECT` в общем случае

    - CTE
- Изменение данных
    - Виртуальные таблицы
    - INSERT
    - UPDATE
    - DELETE
    - RETURNING
    - USING
    - TRUNCATE
- Индексы
    - Цена индексов
    - Обновление индекса
    - B-Tree
    - Составной индекс
    - Определение полей для индексации
- Оптимизация запросов
    - Planner
    - Сканирование таблиц
    - Соединение таблиц
    - Cond
        - Index Cond
        - Filter
        - Hash Cond
        - Merge Cond
        - Join Filter
        - Recheck Cond
    - Поля плана
    - Разбор плана
    - Время выполнения
    - Дополнительные узлы
- Storage
- Vacuum
- WAL
    - Checkpoint
- Locks
    - Уровни блокировок
        - Блокировки на уровне объектов (таблиц)
        - Блокировки на уровне строк (Row-Level Locks)

        - Блокировки на уровне предикатов (Predicate Locks)
        - Блокировки в памяти (Lightweight Locks/Latches)
    - Оптимистичные и пессимистичные блокировки
        - Пессимистичная блокировка
        - Оптимистичная блокировка
        - Summary optimistic vs pessimistic locking
- Партицирование, шардирование и реплицирование
    - Партицирование
        - Отличие от шардирования
        - Типы партицирования в PostgreSQL
            - По диапазону (RANGE)
            - По списку (LIST)
            - По хэшу (HASH)
        - Когда НЕ нужно партицировать
        - Partition pruning
    - Шардирование
    - Реплицирование
        - Типы репликации
            - Физическая репликация
            - Логическая репликация
        - Синхронная vs Асинхронная репликация
        - Что реплицирование НЕ решает
    - Шпаргалка: что когда использовать
- Курсорная пагинация


# Основные понятия

**PostgreSQL** - расширение/диалект языка SQL.

**База данных** - набор структурированных данных

**СУБД** - программа,обеспечивающая доступ к данным, предназначенная для создания, ведения и совместного применения БД многими пользователями.

В СУБД есть процессор языка запросов, который позволяет пользователю вводить команды языка высокого уровня (SQL или др.)

Систему баз данных можно разделить на:

- Сервер = СУБД

- Набор клиентов = приложения, которые обращаются к БД

**Таблицы** состоят из строки и столбцов, на пересечении которых находятся атомарные значения, не разбиваемые без потери смысла.

**Таблицы** называются отношениями.

- Строки - кортежи: количество кортежей = кардинальное число;
- Столбцы - атрибуты: количество атрибутов = степень отношения;

**Ключи** - комбинация атрибутов для связи таблиц (адресации строк)

**Потенциальный ключ** - комбинация атрибутов по которой можно уникальным образом идектифицировать строку в таблице.

Потенциальный ключ должен быть неизбыточным, то есть никакой из атрибутов в нем не должен быть сам по себе уникальным.

Из набора потенциальных ключей один выбирается как первичный ключ (PK), остальные являются альтернативными.

**Внешний ключ (FK)** - атрибут таблицы, ссылающийся на первичный ключ другой таблицы.

Таблица, содержащая FK называется ссылающейся таблицей.

Таблица, содержащая соответствующий PK называется ссылочной таблицей.

FK может быть составным, он не обязан быть уникальным.

Несоответствие данных = проблема ссылочной целостности

Ограичение по соответствию FK значениям PK = ограничения ссылочной целостности.

# SQL язык

Язык запросов SQL - непроцедурный язык работы с СУБД.

Непроцедурный язык (Декларативный) - на котором пишем "что хотим получить". Высокоуровневый.

Процедурный язык (Императивный) - на котором пишем "как решить задачу пошагово". Низкоуровневый.

## Группы операторов

- DDL (Data Definition Language) - определение данных.

- DML (Data Manipulation Language) - манипуляция данными.
- DCL (Data Control Language) - управление правами доступа к объектам БД.

**DDL** - создание, изменение, удаление таблиц, представлений и других объектов БД.

**DML** - выборка, вставка, изменение, удаление строк.

**DCL** - предоставление, отзыв прав.

*Дополнительно*

- TCL (Transaction Control Language) - управление транзакциями.

*Команды*

- `psql` - интерактивный терминал для PostgreSQL.
- `\dt` - просмотр всех таблиц и представлений подключенной БД.
- `\d [table_name]` - просмотр структуры таблицы.
- `\h` - просмотр списка команд PostgreSQL.
- `\h [CREATE TABLE]` - просмотр описания команды CREATE TABLE.

**Кластер** - группа серверов/программ (независимых), работающих вместе как единая система.

**Кластер PostgreSQL** - совокупность БД, управляемых одним экземпляром сервера.

# Операции с таблицами

`[]` - необязательные параметры

## Создание таблиц

```
CREATE TABLE [имя_таблицы] (
  имя_поля тип_данных [ограничение_целостности],
  ...
  имя_поля тип_данных [ограничение_целостности],
  [ограничение_целостности],
  [первичный_ключ],
  [внешний_ключ]
);
```

Пример:

```
CREATE TABLE aircrafts(
  aircraft_code char(3) NOT NULL,
  model text NOT NULL,
  range interger NOT NULL,
  CHECK (range > 0),
  PRIMARY KEY (aircraft_code)
);
```

*Примечание*

- Регистр символов для ключевых слов не важен.
- По умолчанию таблицы создаются в схеме public.

**Схема** - логический фрагмент БД, в котором создаются таблицы (можно воспринимать наверное как namespace).

- Для PK по умолчанию создается B-Tree индекс
- Для `CHECK` , PK, FK и др. автоматически создаются `constraint` по принципу именования: `[таблица]_[колонка]_[тип_ограничения]`

## Удаление таблиц

```
DROP TABLE [имя_таблицы];
```

Пример:

```
DROP TABLE aircrafts;
```

## Вставка данных в таблицу

```
INSERT INTO имя_таблицы
[(имя_атрибута, имя_атрибута, ...)]
VALUES
((значение_атрибута, значение_атрибута, ...),
 (значение_атрибута, значение_атрибута, ...),
 (значение_атрибута, значение_атрибута, ...),
 ...);
```

Пример:

```
INSERT INTO aircrafts
(aircraft_code, model, range)
VALUES
('A1', 'Boeing 737', 3000),
('A2', 'Airbus A380', 4000),
('A3', 'Boeing 747', 5000);
```

*Примечание*

- Можно не указывать имена атрибутов, тогда вставка будет в том же порядке, в котором определны атрибуты при создании таблицы.
- Для строков типов обязательно указывать их в одинарных кавычках, для числовых - нет.

**Результат вставки:** `sql INSERT 0 1`

- Первое число: обычно 0, обозначает OID (Object ID). Раньше можно было найти объект (любой по OID). Сейчас 0 для оптимизации, потому что раньше для каждой строки OID был 4 байта.
- Второе число: количество вставленных строк.

## Выборка данных

```
SELECT [поля]
FROM [таблица]
WHERE [условие];
```

Пример:

```
SELECT aircraft_code, model, range
FROM aircrafts
WHERE range > 3000;
```

*Примечание*

- `SELECT` не гарантирует возвращаемый порядок такой же, как при вставке.

*Упорядочивание:*

```
ORDER BY атрибут_1, атрибут_2, ...
```

*Примечание*

Результирующую выборку можно выводить отсортированной по одному или нескольким параметрам (атрибутам) и переименовывая результирующие столбцы через `AS`.

Пример:

```
SELECT flight_no AS "номер",
  scheduled_dep,
FROM flights
ORDER BY scheduled_dep DESC
LIMIT 5;
```

`LIMIT` - количество возвращаемых строк.

## Выборка определенных строк

```
SELECT *
FROM table
WHERE условие_1 AND условие_2 OR условие_3;
```

## Обновление данных

```
UPDATE имя_таблицы
SET атрибут_1 = значение_1,
    атрибут_2 = значение_2,
    ...
WHERE условие;
```

Пример:

```
UPDATE aircrafts
SET range = range * 1.1
WHERE range > 3000;
```

Результат:

```
UPDATE 1
```

В результате число обновленных строк.

Если под условие подойдет n строк, то все n строк обновятся.

## Ограничение количества строк вывода

```
SELECT ... LIMIT x;
```

Выведет первые x строк.

Чтобы пропустить строки в выводе используется `OFFSET`.

```
SELECT ... OFFSET x;
```

x - сколько первых строк вывода пропустить.

```
DELETE FROM имя_таблицы
WHERE условие;
```

Пример:

```
DELETE FROM aircrafts
WHERE range > 3000;
```

Для удаления всех строк:

```
DELETE FROM имя_таблицы;
```

# Типы данных

## Числовые

| Тип данных | Алиас | Размер | Диапазон | Применение |
|---|---|---|---|---|
| SMALLINT | int2 | 2 bytes | +-32k | Счетчик |
| INTEGER | int, int4 | 4 bytes | +-2,1M | ID |
| BIGINT | int8 | 8 bytes | +-9,2 квинт. | ID/Счетчик |

| Тип данных | Алиас | Размер | Диапазон | Применение |
|---|---|---|---|---|
| DECIMAL | NUMERIC | dynamic | до 131072 цифр | Деньги/Точные рассчеты |
| NUMERIC(AS) | - | dynamic | всего p цифр, s после запятой | Финансы |
| REAL | float4 | 4 bytes | 6 десятичных цифр | Наука/Графика |
| DOUBLE PRECISION | float, float8 | 8 bytes | 15 десятичных цифр | Наука/Рассчеты |
| SMALLSERIAL | - | 2 bytes | [1;32k] | автоинкрементный счетчик |
| SERIAL | - | 4 bytes | [1;2,1 млрд.] | автоинкрементный счетчик |
| BIGSERIAL | - | 8 bytes | [1;9,2 квинт.] | автоинкрементный счетчик |

**Символьные**

| Тип данных | Максимум | Пробелы | Применение |
|---|---|---|---|
| CHAR(n) | n bytes | добавляет | Строго фикксированная длина |
| VARCHAR(n) | n bytes | не добавляет | Текст с известным максимумом |
| TEXT | 1 GB | не добавляет | Любой текст |
| CHAR | 1 символ | - | флаги ('Y', 'N') |

**Дата и время**

| Тип данных | Размер | Точность | Применение |
|---|---|---|---|
| DATE | 4 bytes | день | Дата события |
| TIME | 8 bytes | 1 мкс | Время без даты |
| TIMESTAMP | 8 bytes | 1 мкс | Время с датой без часового пояса |
| TIMESTAMPTZ (приводится к UTC, при выводе подстраивается под TZ | 8 bytes | 1 мкс | Время с датой с часовым поясом |

| Тип данных | Размер | Точность | Применение |
|---|---|---|---|
| пользователя) | | | |
| INTERVAL | 16 bytes | 1 мкс | Продолжительность времени |

**Логические типы**

| Тип | Значение | Размер | Применение |
|---|---|---|---|
| BOOLEAN | true/false | 1 byte | Флаг |

**JSON**

| Тип данных | Скорость | Индексация | Применение |
|---|---|---|---|
| JSON | медленно | нет | когда важен исходный формат (читаемость) |
| JSONB | быстро | GIN, B-Tree | почти всегда, когда нужны запросы к JSON |

**Массивы**

| Тип данных | Пример | Применение |
|---|---|---|
| TEXT[] | {"wifi", "GPS"} | Списки значений |
| INT[] | {1, 2, 3} | Массив целых чисел |
| любой_тип[] | DATE[] | массив дат |

**Сетевые**

| Тип | Пример | Применение |
|---|---|---|
| INET | 192.168.1.1 | IPv4 |
| CIDR | 192.168.1.1/24 | IPv4 |
| MACADDR | 00:11:22:33:44:55 | MAC адрес |

### UUID

| Тип | Размер | Применение |
|---|---|---|
| UUID | 16 bytes | Глобально уникальный ID |

### Бинарные

| Тип | Максимум | Применение |
|---|---|---|
| BYTEA | 1 GB | Файлы |

### Геометрические

| Тип | Описание |
|---|---|
| POINT | Координаты точки (x, y) |
| LINE | Линия |
| POLYGON | Многоугольник |
| CIRCLE | Круг |

# Каскадное удаление

```
CREATE TABLE seats(
  aircraft_code char(3) NOT NULL,
  seat_no varchar(4) NOT NULL,
  PRIMARY KEY (aircraft_code, seat_no),
  FOREIGN KEY (aircraft_code)
    REFERENCES aircrafts(aircraft_code)
    ON DELETE CASCADE
);
```

То есть при удалении записи в таблице `aircrafts` все связанные с ней записи в таблице `seats` также удаляются.

Если не указать `ON DELETE CASCADE`, то все связанные с удаляемой записью записи в таблице `seats` не будут удалены и вернется ошибка.

# Подсчет строк

```
SELECT COUNT(*)
FROM seats
WHERE условие;
```

# Группировка результата

```
SELECT owner_id, status, COUNT(*)
FROM transactions
GROUP BY owner_id, status;
```

Группирует строки по одинаковым параметрам

Если поменять параметры группировки, результат не изменится, так как создает комбинации значений.

*Правило использования GROUP BY*

1. В SELECT можно использовать только столбцы из GROUP BY
2. Агрегатные функции (count, sum, avg и др.)
3. Конструнты
4. Выражения, зависящие только от GROUP BY столбцов

# Агрегатные функции

Агрегатные функции - функции, принимающие колонку или группу строк и возвращающие одно значение.

Обычные функции - принимают строку и возвращают строку.

Пример:

```
SELECT SUM(total_amount) FROM bookings;
```

Пробежит по колонке total_amounts и сложит все значения.

Пример:

```
SELECT
  status,
  SUM(total_amount) AS group_sum
FROM bookings
JOIN tickets t ON b.book_ref = t.book_ref
JOIN tickets_flights tf ON t.ticket_no = tf.flight_id
GROUP BY status;
```

1. Строки разобьются по группам по status:

- группа 1: status1: стобец(str1, str2, ...)
- группа 2: status2: стобец(str1, str2, ...)
- ...

2. К каждой группе применится SUM(total_amount)

- группа 1: status1: (total_amount1)
- группа 2: status2: (total_amount2)
- ...

3. Результат: 1 строка для каждого status

*Примечание*

Если бы не было GROUP BY и нужно было бы посчитать одну и ту же величину для строк с разными значениями какого-то атрибута, например, количество мест в разных самолетах, то пришлось бы делать несколько запросов:

```
SELECT SUM(total_amount) FROM bookings WHERE aircraft_code = 'A1';
SELECT SUM(total_amount) FROM bookings WHERE aircraft_code = 'A2';
SELECT SUM(total_amount) FROM bookings WHERE aircraft_code = 'A3';
```

Но с GROUP BY можно без этого.

То есть GROUP BY позволяет применить аггрегатную функцию к группе строк с одинаковым набором указанных значений атрибутов.

То есть указали plane, class то ко всем строкам с plane_i class_j применится функция и они схлопнутся в одну строку.

# Работа с датами

Получение текущей даты:

```
SELECT CURRENT_DATE; -- <- Current date это функция
SELECT CURRENT_TIME; -- <- Current time это функция
SELECT CURRENT_TIMESTAMP; -- <- Current timestamp это функция
```

Получение даты с точностью:

```
SELECT (date_trunc('hour', CURRENT_TIMESTAMP));
```

Выведет:

| date_trunc |
| --- |
| 2020-01-01 00:00:00+03 |

Извлечение из даты:

```
SELECT EXTRACT('mon' FROM TIMESTAMP '2020-01-01 00:00:00+03');
```

Выведет:

| date_part |
| --- |
| 1 |

Сравнение:

```
SELECT birthday FROM users WHERE
  birthday > '2020-01-01' AND
  birthday < '2020-01-02';
```

# Массивы

Синтаксис записи:

```
ARRAY[1, 2, 3] -- <-конструктор ARRAY
'{1, 2, 3}'::INT[] -- <- строковый литерал '' и приведение типа '{1, 2, 3}' к типу INT[]
ARRAY(SELECT flight_no FROM flights LIMIT 3) -- <-выборка из таблицы flights
```

Приведение типа:

```
'{{1, 2}, {3, 4}}'::INT[][] -- <-строковый литерал '' и приведение типа '{{1, 2}, {3, 4}}' к т
```

### Операции с массивами

Добавленеие/удаление; `||` - конкатенация.

```
SELECT ARRAY[1, 2, 3] || ARRAY[4, 5, 6]; -- результат: [1, 2, 3, 4, 5, 6]
SELECT ARRAY[1, 2, 3] || 4; -- результат: [1, 2, 3, 4]
SELECT 0 || ARRAY[4, 5, 6]; -- результат: [0, 4, 5, 6]
array_append(ARRAY[1, 2], 3) -- результат: [1, 2, 3], добавить в конец
array_prepend(ARRAY[2, 3], 1) -- результат: [1, 2, 3], добавить в начало
array_remove(ARRAY[1, 1, 2, 1], 1) -- результат: [2], удалить все вхождения
```

Также можно удалять через срезы кастомной функцией:

```
CREATE OR REPLACE FUNCTION array_remove(arr ARRAY, idx INT)
RETURNS ARRAY AS $$
BEGIN
  RETURN arr[1:idx-1] || arr[idx+1:array_length(arr, 1)];
END;
$$ LANGUAGE plpgsql;
```

Тогда:

```
SELECT array_remove(ARRAY[1, 2, 3, 4, 5], 3); -- результат: [1, 2, 4, 5]
```

### Методы массивов

```
array_length(arr, int) -- длина массива
array_lower(arr, int) -- нижняя граница массива
array_upper(arr, int) -- верхняя граница массива
array_position(arr, elem) -- позиция элемента в массиве
array_positions(arr, elem) -- массив позиций элемента в массиве
array_replace(arr, elem, new_elem) -- заменить элемент на новый
array_append/prepend/remove -- добавление, удаление элементов
unnest(arr) -- развернуть строки
```

**Сравнение массивов** Через `<`, `>`, `=` в лексикографическом порядке.

### Специальные операторы

Содержание: `@>`, `@<` - содержится ли arr1 в arr2.

Пересечение: `&&` - есть ли пересечение arr1 и arr2.

Сравнение с ANY и ALL:

```
SELECT 3 = ANY(ARRAY[1, 2, 3]); -- результат: true (есть ли элемент 3 в массиве)
SELECT 3 > ANY(ARRAY[1, 2, 3]); -- результат: true (есть ли элементы меньше 3 в массиве)
SELECT 3 < ANY(ARRAY[1, 2, 3]); -- результат: false (есть ли элементы больше 3 в массиве)
SELECT 5 > ALL(ARRAY[1, 2, 3]); -- результат: true (все ли элементы массива < 5)
SELECT 5 < ALL(ARRAY[1, 2, 3]); -- результат: false (все ли элементы массива > 5)
```

**Доступ по индексу**

Индексация массивов с 1.

`arr[i]` - i-тый элемент массива.

`arr[i:j]` - все элементы массива с i по j.

# JSON и JSONB

| | JSON | JSONB |
|---|---|---|
| сохранение | быстрее, так как не анмаршалится | медленнее, так как разбирается при сохранении |
| использование | медленнее, так как каждый раз разанмаршалится | быстрее, так как уже заанмаршален |
| порядок ключей | сохраняет, потому что не анмаршалится | не сохраняет, так как уже заанмаршален |

**Обращение по ключу:**

```
CREATE TABLE pilot_hobbies (
  pilot_name TEXT,
  hobbies JSONB
);

SELECT pilot_name, hobbies->'sports' AS music
FROM pilot_hobbies
WHERE hobbies->'sports' @>'["хоккей"]'::JSONB;
```

**Обновление:**

```
UPDATE pilot_hobbies
SET hobbies = hobbies || '{"sports":["хоккей"]}'
WHERE pilot_name = 'Boris';
```

```
UPDATE pilot_hobbies
SET hobbies = json_set(hobbies, '{sports, 1}', "футбол")
WHERE pilot_name = 'Boris';
```

`{sports, 1}` - в какое место добавить.

Итого:

| pilot_name | hobbies |
|---|---|
| Boris | {"trips": 0, "sports":["хоккей", "футбол"], home_lib: true} |

# Constraint

**Constraint** - ограничение

Создание ограничения:

```
CONSTRAINT имя условие;
```

`CHECK` - условие для атрибутов таблицы.

Для атрибута:

```
CREATE TABLE A(
  name text CHECK(char_length(name) > 3),
);
```

Для таблицы:

```
CREATE TABLE A(
  mark INT
  CONSTRAINT valid_mark CHECK(mark > 0)
);
```

`NOT NULL` - значение обязаны передать или настроить `DEFAULT`

`UNIQUE` - ограничение уникальности на значения в столбце или на комбинацию столбцов.

```
CREATE TABLE A(
  nickname TEXT,
  friends_code text,
```

```
  UNIQUE (nickname, friends_code) -- в таблице каждая пара nickname, friends_code уникальна
);
```

*Примечание*

- На ограничение `UNIQUE` всегда создается `B-Tree` индекс.
- `UNIQUE NOT NULL <=> PK =>` создается `B-Tree` индекс по умолчанию.

Внешний ключ (FK):

```
CREATE TABLE A(
  record INT REFERENCES students,
  ...
);
```

или явно указать ограничение:

```
CREATE TABLE A(
  num INT,
  ...
  FOREIGN KEY (num) REFERENCES students(field)
);
```

**Подходы к удалению строк**

Подходы к удалению строк из главных таблиц при наличии ссылок на них в зависимых:

**1 Каскадное удаление**

При создании зависимой таблицы:

```
CREATE TABLE tab(
  rb INT,
  ...
  FOREIGN KEY (rb) REFERENCES students(record_book)
  ON DELETE CASCADE
);
```

При удалении записи в students, на которую ссылаются записи из tab, эти записи будут удалены.

**2 Запрет на удаление**

Можно запретить удаление из основной таблицы, если в зависимой есть хотя бы одна строка, которая ссылается на запись в основной таблице:

```
CREATE TABLE tab(
  rb INT,
  ...
  FOREIGN KEY (rb) REFERENCES students(record_book)
  ON DELETE RESTRICT # или NO ACTION
);
```

Тогда выведется сообщение об ошибке, если есть хотя бы одна строка.

### 3 Присваивание NULL

При удалении можно присвоить NULL в колонке, если это не нарушает ограничение целостности:

```
CREATE TABLE tab(
  rb INT, -- тут нельзя указать NOT NULL, иначе при удалении сломается
  ...
  FOREIGN KEY (rb) REFERENCES students(record_book)
  ON DELETE SET NULL
);
```

### 4 Присваивание DEFAULT значения

При удалении можно присвоить значение по умолчанию:

```
CREATE TABLE tab(
  rb INT DEFAULT 12345, -- если не указать rb при вставке, то будет записано 12345
  ...
  FOREIGN KEY (rb) REFERENCES students(record_book)
  ON DELETE SET DEFAULT
);
```

### 5 ВАЖНО!!!

При `UPDATE` применяются те же подходы:

```
CREATE TABLE tab(
  rb INT,
  ...
  FOREIGN KEY (rb) REFERENCES students(record_book)
  ON UPDATE CASCADE
);
```

# Выбор схемы и удаление таблиц

### Выбор схемы

```
SET search_path TO schema_name;
```

### Удаление таблиц

```
DROP TABLE имя_таблицы;
```

Если есть ссылки на таблицу, то:

```
DROP TABLE имя_таблицы CASCADE;
```

Если заранее не знаешь, есть ли таблица или нет, то:

```
DROP TABLE IF EXISTS имя_таблицы CASCADE;
```

## Про схемы

Еще раз:

**Схема** - "пространство имен", логический фрагмент БД.

По умолчанию создается схема `public` .

Имена схем должны быть уникальны для одной БД.

Имена таблиц должны быть уникальны внутри схемы.

Для установки схемы для выполнения запросов используют:

```
SET search_path = [имя_схемы_1], [имя_схемы_2], ...;
```

Или для запросов схему можно указать через `.` :

```
SELECT * FROM [имя_схемы_1].[имя_таблицы];
```

Узнать текущую схему можно через:

```
SHOW search_path;
```

# Изменение таблиц

Изменить таблицу:

```
ALTER TABLE имя_таблицы действие;
```

Действия: со столбцами, ограничениями, функциями.

### Добавление столбца

```
ALTER TABLE table_name ADD COLUMN column_name constraints;
```

Пример:

```
ALTER TABLE student ADD COLUMN phone text NOT NULL;
```

### Удаление столбца

```
ALTER TABLE table_name DROP COLUMN column_name;
```

Пример:

```
ALTER TABLE student DROP COLUMN phone;
```

### Изменение столбца

```
ALTER TABLE table_name
  ALTER COLUMN column_name SET DATA TYPE data_type,
  ALTER COLUMN column_name SET DEFAULT default_value,
  ALTER COLUMN column_name DROP DEFAULT,
  ALTER COLUMN column_name SET NOT NULL,
  ALTER COLUMN column_name DROP NOT NULL,
  ...;
```

Некоторые ограничения дропаются только по названию:

```
ALTER TABLE studs
ADD CHECK (age>18);

ALTER TABLE studs
DROP CONSTRAINT studs_age_check;


ALTER TABLE studs
ADD CONSTRAINT pk_uid PRIMARY KEY (id);

ALTER TABLE studs
ADD UNIQUE (name);
```

*Примечание*

Если при изменении таблицы нарушится целостность, то изменения не применятся.

Например, если:

```
ALTER TABLE students ADD COLUMN phone TEXT NOT NULL;
```

То изменения не применятся, так как у нас добавится `NOT NULL` колонка, а значений там нет.

То есть сначала надо:

1

```
ALTER TABLE students ADD COLUMN phone TEXT;
```

2

```
UPDATE students SET phone = '123456789' WHERE phone IS NULL;
```

3

```
ALTER TABLE students ALTER COLUMN phone SET NOT NULL;
```

**Изменение типа столбца**

```
ALTER TABLE students
ALTER COLUMN phone
SET DATA TYPE varchar(10);
```

Изменение типа без дополнительных махинаций сработает только если типы из одной группы.

Если нет, то:

```
ALTER TABLE students
DROP CONSTRAINT {ограничение, которое сломает новый тип};
ALTER COLUMN phone_type
SET DATA TYPE INTEGER
USING (CASE
  WHEN phone_type = 'домашний' THEN 1
  WHEN phone_type = 'рабочий' THEN 2
  ELSE 3
END
);
```

### Переименование столбца

```
ALTER TABLE students RENAME COLUMN phone TO phone_number;
```

### Переименование CONSTRAINT

```
ALTER TABLE students
RENAME CONSTRAINT school_id_fkey
TO studs_school_id_fkey;
```

# Представления

**Представление (VIEW)** - сохраненный запрос, к которому можно обращаться как к таблице.

`VIEW` - объект БД, хранящий SQL определение запроса.

При использовании `VIEW` данные вычисляются заново.

`MATERIALIZED VIEW` - сохраненный результат запроса, который физически записан на диске и читается как готовые данные.

При использовании `MATERIALIZED VIEW` данные берутся из сохраненного при создании результата.

Если данные обновились, то нужно рефрешить `MATERIALIZED VIEW`, иначе на диске будут храниться неактуальные данные.

```
REFRESH MATERIALIZED VIEW имя_материализованного_view;
```

Если нужно изменить запрос, используется:

```
CREATE OR REPLACE VIEW имя
```

Если изменились названия столбцов, то нужно сначала удалить представление, иначе будет ошибка.

```
DROP VIEW seats_by_fare_cond;
CREATE OR REPLACE VIEW seats_by_fare_cond AS
SELECT
  seat_no,
  fare,
  COUNT(*) AS cnt
FROM seats
GROUP BY seat_no, fare
ORDER BY seat_no, fare;
```

# Аномалии и транзакции

**Транзакция** - набор операций/действий, которые выполняются либо вместе, либо никак.

**Аномалии** - ситуации, которые приводят к противоречиям данных или усложняют обработку.

## ACID

**ACID** - набор ключевых свойств транзакций в реляционных БД.

**A** - **Atomicity** - атомарность **C** - **Consistency** - целостность **I** - **Isolation** - изоляция **D** - **Durability** - постоянность

### Atomicity

**Принцип:**

Выполняется либо все, либо ничего

Пример:

```
BEGIN
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

Либо оба апдейта закоммитятся, либо оба отменятся.

## Consistency

**Принцип:**

Транзакция сохраняет целостность данных.

Транзакция переводит таблицу из одного корректного состояния в другое корректное состояние.

То есть после `COMMIT` сохраняются все:

- `PK`
- `UNIQUE`
- `FK`
- `CONSTRAINT`
- бизнес инварианты

Пример:

Если в БД до транзакции было `CHECK (balance >= 0)`, то после любой транзакции не получится что в таблице окажется запись с `balance < 0`.

То есть БД не дает закоммитить некорректное состояние и делает `ROLLBACK`.

## Isolation

Изоляция определяет, насколько одна транзакция видит изменения другой (незакомиченные или параллельные) и какие аномалии допустимы.

То есть результат параллельного выполнения двух транзакций должен быть предсказуемым и соответствовать уровню изоляции.

Изоляция нужна чтобы параллельные транзакции не мешали друг другу и чтобы избежать аномалии.

## Durability

**Принцип:**

Если транзакция закоммичена, то ее результат не должен потеряться, даже если после этого произошел сбой.

То есть после `COMMIT` транзакция должна пережить:

- падение процесса

- рестарт сервера БД
- сбой приложения

Это обеспечивается через:

- WAL / журнал транзакций
- механизмы восстановления после сбоя
- fsync

## Аномалии

Основные аномалии:

- Dirty read
- Non-repeatable read
- Phantom read
- Lost update

### Dirty read

**Определение:**

Грязное чтение - чтение данных, которая другая транзакция еще не закоммитила.

**Пример:**

```
-- T1

BEGIN;
UPDATE accounts SET balance = 0 WHERE id = 1;
-- пока не COMMIT

-- T2
BEGIN;
SELECT balance FROM accounts WHERE id = 1;
-- увидит 0, вместо старого значения
COMMIT;
```

Если T1 сделает `ROLLBACK`, то T2 прочитала данные, которые могли никогда не стать реальными.

### Non-repeatable read

**Определение:**

Неповторяемое чтение - одна и та же транзакция читает одну и ту же строку и получает разные значения, потому что другая транзакция уже закоммитила изменения.

**Пример:**

```
-- T1

BEGIN;
SELECT balance FROM accounts WHERE id = 1; -- 100
--... в этом промежутке T2 закоммитила свои изменения
SELECT balance FROM accounts WHERE id = 1; -- уже 50
COMMIT;

-- T2

BEGIN;
UPDATE accounts SET balance = 50 WHERE id = 1;
COMMIT;
```

## Phantom read

**Определение:**

Фантомное чтение - при выполнении одного и того же запроса на чтение по условию во второй раз получу другой набор строк, потому что кто-то параллельно вставил/удалил записи.

**Пример:**

```
-- T1

BEGIN;
SELECT COUNT(*) FROM orders WHERE status = 'new'; -- допустим 10 строк
-- ... в этот момент T2 добавила строку
SELECT COUNT(*) FROM orders WHERE status = 'new'; -- уже 11 строк
COMMIT;

-- T2

BEGIN;
INSERT INTO orders (status) VALUES ('new');
COMMIT;
```

## Lost update

**Определение:**

Потерянное обновление - две транзакции читают одно и то же, потом обе обновляют и одно значение затирает другое.

**Пример:**

```
-- T1
BEGIN;
SELECT balance FROM accounts WHERE id = 1; -- 100
-- решаем прибавить 10
UPDATE accounts SET balance = balance + 10 WHERE id = 1;
COMMIT;

-- T2
BEGIN;
SELECT balance FROM accounts WHERE id = 1; -- 100
-- решаем прибавить 20
UPDATE accounts SET balance = balance + 20 WHERE id = 1;
COMMIT;
```

Итог должен быть 130, а стал или 110, или 120.

## Serialization anomaly

**Определение:**

Аномалия, которая возникает даже если защитились от non-repeatable read, dirty read, phantom read, но получили некорректный результат, из-за того, что несколько транзакций проверили одно и то же условие и принимали решения на основе одного и того же снимка данных.

**Пример:**

Допустим есть правило, что хотя бы один врач должен быть на смене.

doctors_on_call:

| doctor_id | on_call |
|---|---|
| 1 | true |
| 2 | true |

Запускаются параллельные транзакции, которые видят, что два врача на смене, значит одному можно уйти.

```
-- T1
BEGIN;
SELECT COUNT(*) FROM doctors_on_call
WHERE on_call = true; -- 2
UPDATE doctors_on_call SET on_call = false
WHERE doctor_id = 1;
COMMIT;
```

```
-- T2
BEGIN;
SELECT COUNT(*) FROM doctors_on_call
WHERE on_call = true; -- 2
UPDATE doctors_on_call SET on_call = false
WHERE doctor_id = 2;
COMMIT;
```

Результат:

doctors_on_call:

| doctor_id | on_call |
|---|---|
| 1 | false |
| 2 | false |

То есть каждая транзакция увидела, что все норм, но в итоге некорректный результат.

## Уровни изоляции

Уровени изоляции опрделеяют, какие аномалии мы допускаем.

Основные уровни изоляции в порядке увеличения строгости:

1. Read Uncommitted
2. Read Committed
3. Repeatable Read
4. Serializable

### Read Uncommitted

**Определение:**

Самый слабый уровень изоляции.

Разрешает Dirty Read (в PostgreSQL нет)

Используется там, где важна максимальная скорость и допустимы грязные данные.

### Read Committed

**Определение:**

Каждый отдельный SQL Statement (не вся транзакция) видит данные, которые были закоммичены на момент его начала.

Не допускает Dirty Read, но допускает остальные аномалии

**ВАЖНО**

Уровень изоляции по умолчанию в PostgreSQL

Snapshot делается на каждый отдельный SQL Statement

### Repeatable Read

**Определение:**

Вся транзакция видит один snapshot данных, который был в момент начала транзакции. То есть внутри транзакции видим данные так, как будто параллельные ничего не меняли.

Не допускает Dirty Read и Non-repeatable Read, но допускает оставшиеся аномалии.

**ВАЖНО**

Snapshot создается один раз на всю транзакцию

### Serializable

**Определение:**

Самый строгий уровен изоляци.

Результат будет такой, как будто транзакции выполнились последовательно.

Не допускает основные аномалии.

## MVCC (Multi-Version concurrency control)

**MVCC** - многоверсионная модель.

Она предполагает, что каждый SQL-оператор видит так называемый снимок данных (snapshot), то есть согласованное состояние (версию) базы данных, которое она имела на определенный момент времени.

При этом параллельно исполняемые транзакции, даже вносящие изменения в базу данных не нарушают согласованности данных этого снапшота.

**Snapshot** - это не просто копия всех данных, а скорее правило видимости.

Он говорит:

- Какие транзакции уже были завершены?
- Какие ещё выполнялись?
- Какие изменения можно видеть?
- Какие изменения ещё нельзя видеть?

**Основная идея**

PostgreSQL обычно не перезаписывает строку на месте. Вместо этого он создаёт новую версию строки (физически прямо в таблице), а старую оставляет, пока она может быть нужна другим транзакциям.

Каждая транзакция сама решает, какую из этих версий ей можно видеть (зависит от уровня изоляции).

При этом обычные чтения не блокируют обычные записи, обычные записи не блокируют обычные чтения.

Блокировки как раз используются при транзакциях.

**КАК Я ЭТО ПОКА В ГОЛОВЕ УЛОЖИЛ**

**MVCC** — это многоверсионная модель контроля конкурентного доступа. В PostgreSQL при изменении строки старая версия обычно не перезаписывается, а создаётся новая физическая версия строки. Старая версия остаётся в таблице и помечается как заменённая или удалённая. Каждая транзакция работает со своим snapshot и по нему решает, какие версии строк ей видны. После завершения транзакций старые версии, которые уже никому не нужны, очищает VACUUM. Благодаря этому обычные чтения не блокируют обычные записи, а записи не блокируют чтения.

То есть основная фишка MVCC, что из-за нее много транзакций конкурентно могут читать и писать данные (это хорошо для нагрузки).

То есть при `UPDATE` у меня не физически заменится строка, а появится рядом новая версия этой строки.

При этом PostgreSQL хранит информацию о версиях строк, то есть:

Пусть была строка в таблице `accounts` с `balance=100`:

```
id = 1, balance = 100
создана транзакцией 10
удалена/заменена: нет
```

Потом я сделал:

```
UPDATE accounts
SET balance = 200
WHERE id = 1;
```

Старая версия при этом не перетерлась, а рядом положилась еще версия этой строки:

```
Версия A:
id = 1, balance = 100
создана транзакцией 10
заменена транзакцией 20

Версия B:
id = 1, balance = 200
создана транзакцией 20
заменена: нет
```

То есть грубо `UPDATE` можно представить как:

- пометить старую версию как устаревшую
- создать новую версию строки

Для каждой версии строки PostgreSQL хранит служебную информацию, упрощенно: `xmin` - какая транзакция создала эту версию `xmax` - какая транзакция удалила или заменила эту версию

Если `xmax = 0`, то это актуальная версия строки.

```
id | balance | xmin | xmax
---+---------+------+------
1  | 100     | 10   | 20
1  | 200     | 20   | 0
```

То есть при старте транзакции она будет выбирать в зависимости от уровня изоляции, какую версию строки использовать.

Допустим будет две транзакции:

- одна с `READ COMMITTED`, она будет видеть актуальную версию строки, то есть если уже какая-то транзакция закоммитила 200, то следующий селект текущей транзакции увидит 200.
- одна с `REPEATABLE READ`, она будет видеть версию строки на момент запуска, то есть 100, и даже несмотря на то, что уже другая транзакция закоммитила 200 и та, которая выше описанга прочитала 200, эта до сих пор будет видеть версию строки с 100.

При `DELETE` строка помечается как удаленная, затем `VACUUM` ее удалит физически.

При этом при `SELECT` PostgreSQL фильтрует версии через MVCC (то есть смотрит, какие транзакции уже завершились, какие выполняются, какие изменения можно видеть), поэтому мертвые строки не берутся, а для существующих берется актуальная версия.

При этом версия строки видна, только если:

- транзакция, которая ее создала уже `COMMIT`
- версия не была удалена/заменена другой закоммиченной транзакцией до момента snapshot
- версия не создана "будущей" транзакцией, которая началась после snapshot

При `INSERT` если даже произошел `ROLLBACK` , новая физическая строка не будет видна, какое то время будет лежать в таблице, но будет считаться невидимой.

**ВАЖНО**

Долгие транзакции как раз вредны тем, что MVCC не помечает версии строк как ненужные, из-за этого `VACUUM` думает, что эти строки еще нужны другим транзакциям и не может их удалить.

Из-за этого растет размер таблиц, индексы (bloat - раздувание), запросы могут становиться медленнее.

**ВАЖНО**

При изменении строк по идее должна добавляться запись в индекс, но старая запись в индексе тоже не исчезает сразу, так как может быть видна старая версия строки другим транзакциям.

Есть оптимизация `HOT Update (heap-only tuple)` :

если постгрес может обновить строку без изменения индексов, он создает строки только в таблице, а индексные записи не трогает.

Он свяжет старый индекс с новой версией строки.

## Блокировки транзакций

# Дополнительно про запросы

## LIKE и NOT LIKE

Поиск по шаблонам.

Если его использовать, то index по столбцу не будет работать.

`%` - любая полседовательность символов.

То есть:

| шаблон | вывод |
| --- | --- |
| %имя | 1 имя , abc имя , имя , ... |
| имя% | имя 1 , имя abc , имя , ... |
| %имя% | 1 имя , abc имя , имя , ... |

`_` - один любой символ

То есть если нужно найти все строки длины 3:

```
SELECT * FROM A where name ILIKE '___';
```

Для регулярок POSIX используют `~'{регулярное выражение}'` .

```
SELECT * FROM A where name ~ '^[a-z]+$';
```

**Вычисляемые столбцы**

При выборке можно выбирать выисляемые столбцы:

```
SELECT model, rabge round(range/1.609, 2) AS miles;
```

# Условная конструкция

```
CASE WHEN условие THEN выражение
  [WHEN условие THEN выражение]
  ...
  [ELSE выражение]
END
```

Например, для выборки:

```
SELECT model, range,
  CASE WHEN range < 2000 THEN 'Ближний'
```

```
        WHEN range >= 2000 AND range < 3000 THEN 'Средний'
        ELSE 'Дальний'
    END AS range_type
FROM aircrafts;
```

# Join'ы

**Join'ы** - соединения таблиц.

## INNER JOIN / JOIN

**Join** - склеить все строки левой со строками в правой, для которых есть строки в правой.

A:

| id | param_id |
|---|---|
| 1 | a |
| 2 | b |

B:

| param | name |
|---|---|
| a | hello |
| b | world |
| c | ! |

```
SELECT * FROM a A JOIN b B ON a.param_id = b.param;
```

Результат:

| id | param_id | name |
|---|---|---|
| 1 | a | hello |
| 2 | b | world |

**CROSS JOIN** - декартово произведение таблиц:

```
SELECT * FROM a A CROSS JOIN b B;
```

Результат:

| id | param_id | name | param |
|---|---|---|---|
| 1 | a | hello | a |
| 1 | a | world | b |
| 1 | a | ! | c |
| 2 | b | hello | a |
| ... | ... | ... | ... |

Всего будет 2*3 = 6 строк.

## LEFT JOIN

Оставляем все строки левой таблицы.

Для строк левой, у которых нет строки в правой все столбцы правой будут заполнены NULL.

*class_info*

| id | name |
|---|---|
| 1 | 10A |
| 2 | 10B |
| 3 | 10C |

*students*

| name | id | class_id |
|---|---|---|
| Vlad | 123 | 1 |
| Tima | 456 | 2 |
| Dan | 789 | 1 |

```
SELECT * FROM class_info c
LEFT JOIN students s
ON s.class_id = c.id;
```

Результат:

| id | c.name | s.name | id |
|---|---|---|---|
| 1 | 10A | Vlad | 123 |
| 2 | 10B | Tima | 456 |
| 3 | 10C | NULL | NULL |
| 1 | 10A | Dan | 789 |

## RIGHT JOIN

Оставляем все строки правой таблицы.

Зеркало Left Join.

*vacancy*

| id | promo_id |
|---|---|
| 1 | ccee |
| 2 | aabb |
| 3 | NULL |

*promos*

| id | name |
|---|---|
| aabb | promo1 |
| ccdd | promo2 |
| eeff | promo3 |

```
SELECT * FROM vacancy v
RIGHT JOIN promos p
ON v.promo_id = p.id;
```

Результат:

| p.id | p.name | v.id |
|---|---|---|
| aabb | promo1 | 2 |

| p.id | p.name | v.id |
|---|---|---|
| ccdd | promo2 | 1 |
| eeff | promo3 | NULL |

**Примечание**

Пересеч таблицы можно и без JOIN, а через условие WHERE:

```
SELECT a.col1, b.col2 FROM a A, b B
WHERE a.col3 = b.col3;
```

Можно также соединить таблицы сами с собой:

Например, сколько путей между всеми аэропортами в таблице можно составить:

```
SELECT COUNT(*)
FROM airports a1, airports a2
WHERE a1.city != a2.city;
```

Или

```
SELECT COUNT(*)
FROM airports a1
JOIN airports a2
ON a1.city != a2.city;
```

Или

```
SELECT COUNT(*)
FROM airports a1
CROSS JOIN airports a2
WHERE a1.city != a2.city;
```

## OUTER JOIN

Сохраняет строки без пары

Для Left и Right результаты не меняются, то есть:

- Left Join = Left Outter Join
- Right Join = Right Outter Join

Для Inner Join = Jin результаты дополняются строками без пары:

То есть:

Inner Join - только пересечение A и B

FULL OUTTER JOIN - пересечение A и B + остатки A + остатки B

# Множества

В SELECT предусмотрены средства выполнения операций с выборками, как с множествами.

- `UNION` для вычисления объединения множеств строк из двух выборок;
- `INTERSECT` для вычисления пересечения множеств строк из двух выборок;
- `EXCEPT` для вычисления разности множеств строк из двух выборок.

То есть это операции, которые используются для работы с наборами строк, которые вернули SELECT, как с множествами.

- `UNION` - объединение
- `INTERSECT` - пересечение
- `EXCEPT` - разность

Основное правило:

У всех SELECT должно быть одинаковое количество колонок и совместимые типы, то есть:

Такой кейс сломается:

```
SELECT id, email FROM users
UNION
SELECT id FROM admins;
```

А такой кейс нет:

```
SELECT id, email FROM users
UNION
SELECT id, email FROM admins;
```

Названия колонок берется из первого SELECT.

## UNION

Вычисляет объединение множеств строк из двух выборок и убирает дубликаты.

Пример:

```
SELECT email FROM users
UNION
SELECT email FROM admins;
```

Вернет все email из обеих таблиц без дубликатов.

Но если мне нужно поселектить из UNION, его нужно вынести в CTE или подзапрос.

Если я селекчу `uid, emails из users, admins` , и есть чувак с одним и тем же email, но разным id в таблицах, то отдаст две строки, так как уберат полные дубликаты по всем атрибутам.

То есть дубликаты убираются по всем атрибутам, по которым я селекчу.

**Пример 1**

Пусть у меня:

*client*

| uid | email | name |
|---|---|---|
| 1 | a@a.com | a |
| 2 | b@b.com | b |

*admins*

| id | email | name | role |
|---|---|---|---|
| 123 | a@a.com | a | admin |
| 456 | c@c.com | c | super-admin |

```
WITH emails AS (
  SELECT email from client
  UNION
  SELECT email from admins
)
SELECT email FROM emails;
```

Результат:

| email |
| --- |
| a@a.com |
| b@b.com |
| c@c.com |

Тут из-за того, что мы селектим только email, убрался дубликат для a@a.com.

**Пример 2**

Попробуем из той же таблицы заселектить email и id:

```
WITH emails AS (
  SELECT uid, email, 'cl' FROM client
  UNION
  SELECT id, email, 'adm' FROM admins
)
SELECT * FROM emails;
```

Результат:

| uid | email | col? |
| --- | --- | --- |
| 1 | a@a.com | cl |
| 2 | b@b.com | cl |
| 123 | a@a.com | adm |
| 456 | c@c.com | adm |

Тут у нас (1, a@a.com) != (123, a@a.com), соответственно это не дубликат

**UNION ALL** - то же самое, только дубликаты не удаляются.

Из-за этого `UNION ALL` работает быстрее, чем `UNION` .

# INTERSECT

Возвращает строки, которые есть и в первом, и во втором запросе.

`INTERSECT` возвращает уникальные значения и убирает дубликаты.

`INTERSECT ALL` возвращает уникальные значения с дубликатами.

**Пример 1**

*users*

| email |
| --- |
| a@a.com |
| b@b.com |
| c@c.com |

*admins*

| email |
| --- |
| a@a.com |
| b@b.com |
| a@a.com |
| a@a.com |

```
SELECT email FROM users
INTERSECT
SELECT email FROM admins;
```

Результат:

| email |
| --- |
| a@a.com |
| b@b.com |

Тут у нас убрались все дубликаты для a@a.com.

**Пример 2**

Теперь с `INTERSECT ALL`:

```
SELECT email FROM users
INTERSECT ALL
SELECT email FROM admins;
```

Результат:

| email |
| --- |
| a@a.com |
| a@a.com |
| b@b.com |

Тут у нас две записи для a@a.com, то есть `min(count(a@a.com в users), count(a@a.com в admins))` = 2.

Потому что `a@a.com` вернулось min(2, 3) = 2 раза.

## EXCEPT

Вычисляет строки из первого запроса, которых нет во втором запросе.

`EXCEPT` убирает дубликаты.

`EXCEPT ALL` не убирает дубликаты.

**Пример 1**

A: 1, 1, 1, 2 B: 1, 1, 3

`A EXCEPT B` = 2

**Пример 2**

A: 1, 1, 1, 2 B: 1, 1, 3

`A EXCEPT ALL B` = 1, 2

Потому что из трех 1 вычли две 1.

## Замена запросов с UNION, INTERSECT, EXCEPT

Все эти операции можно заменить подзапросами с EXISTS

Если провести аналогию, то `UNION`, `INTERSECT`, `EXCEPT` работают вертикально по атрибутам, а `JOIN` горизонтально.

# Агрегирование и группировка

Агрегатные функции нужно использовать с `GROUP BY`

Для фильтрации групп можно использовать `HAVING`

**Пример**

```
WITH emails AS (
  SELECT email FROM client
  UNION ALL
  SELECT email FROM admins
)
SELECT email, COUNT(*) as cnt
FROM emails
GROUP BY email
HAVING COUNT(*) >= 2; -- отсеиваем группы, где у пользователя < 2 email; используем COUNT(*) в
```

Агрегатные функции с `GROUP BY` вычисляют результат для строк внутри группы.

Если нет `GROUP BY`, то для всех строк в таблице.

# Оконные функции

**Оконная функция** в SQL - функция, которая работает с выделенным набором строк (окном, партицией) и выполняет вычисления для этого набора строк в отдельном столбце.

**Партиции (окна из наборов строк)** - это набор строк, указанный для оконной функции по одному или нескольким столбцам таблицы.

Партиции для каждой оконной функции в одном запросе могут быть разделены по разным столбцам таблицы.

```
  Имя      Предмет       Оценка
+ Маша     Математика    3
|
+ Маша     Русский       4

- Олег     Математика    4
|
- Олег     Русский       5
|
- Олег     Физика        2
```

В данном случае две строки с Машей и три строки с Олегом - партиции оконной функции по столбцу 'Имя'.

Оконные функции в отличие от агрегатных функций с `GROUP BY` не сокращают количество строк в результате (не схлопывают в одну строку).

**Пример**

С агрегатной функцией с GROUP BY:

```
  Имя      Оценка средняя
  Маша     3.5
  Олег     3.66
```

Через оконную функцию:

```
    Имя      Предмет       Оценка  Средняя оценка
  + Маша     Математика    3        3.5
  |
  + Маша     Русский       4        3.5

  - Олег     Математика    4        3.66
  |
  - Олег     Русский       5        3.66
  |
  - Олег     Физика        2        3.66
```

# Порядок выполнения

1. Выборка таблиц, все подзапросы под командой `FROM`
2. Условия фильтрации `WHERE` , группировки `GROUP BY` , `HAVING`
3. Только после этого применяется команда выборки `SELECT` и рассчет оконных функций под выборкой
4. `ORDER BY` всего результата

*Примечание*

Окна создаюбтся после разделения на группы `GROUP BY` если он есть

# Синтаксис оконных функций

```
func_name(column_name) [FILTER (WHERE ...)]
OVER (
  PARTITION BY (column_name_1, column_name_2, ...)
  ORDER BY (column_name_1, column_name_2, ...)
  [frame_clause]
)
```

*Примечание*

`FILTER` работает только с агрегатными функциями.

## Классы оконных функций

| Aggregate | Ranking | Value |
|---|---|---|
| AVG() | DENSE_RANK() | FIRST_VALUE() |
| COUNT() | NTILE() | LAST_VALUE() |
| MAX() | RANK() | LAG() |
| MIN() | ROW_NUMBER() | LEAD() |
| SUM() | CUME_DIST() | NTH_VALUE() |

**Пример**

*feed*

| id | status | event_type |
|---|---|---|
| 1 | approved | registration |
| 2 | rejected | registration |
| 3 | accepted | transaction |
| 4 | rejected | transaction |
| 5 | approved | personal_data |
| 6 | approved | registration |
| 7 | rejected | registration |
| 8 | accepted | transaction |
| 9 | accepted | transaction |

```
SELECT id, status, event_type, ROW_NUMBER()
-- разбиваю таблицу на окна по event_type, потом по status
-- затем внеутри каждого окна применяю ROW_NUMBER()
OVER (PARTITION BY event_type, status) as rn
FROM feed ORDER BY event_type;
```

```
           event_type     status     id      rn
         |-registration   approved   1       1
1 окно   |
         |-registration   approved   6       2

         |-registration   rejected   2       1
2 окно   |
         |-registration   rejected   7       2

         |-transaction    accepted   3       1
         |
3 окно   |-transaction    accepted   8       2
         |
         |-transaction    accepted   9       3

4 окно   |-transaction    rejected   4       1
```

То есть оконные функции работают схоже с агрегатными + `GROUP BY`, но вместо этого мы через `PARTITION BY` разбиваем (~группируем) на окна, заем через `ORDER BY` опционально сортируем данные внутри каждого окна. Затем к каждому окну применяем функцию.

Если же в запросе есть `GROUP BY` + `HAVING`, то оконные функции вызываются после них.

Также можно задать разбиение через `WINDOW` и вызвать функцию для него:

```
SELECT
  id,
  FIRST_VALUE(latitude) OVER tz as first_in_tz
  RANK() OVER tz as rn
FROM table
WINDOW tz AS(PARTITION BY timezone ORDER BY latitude DESC)
ORDER BY timezone, rank;
```

# DISTINCT

`DISTINCT` и `DISTINCT ON` нужны, чтобы убрать дубликаты из результата запроса.

`DISTINCT` убирает все полные дубликаты, то есть сравнивает все атрибуты строк.

`DISTINCT ON` убирает дубликаты только по указанным атрибутам и оставляет первую строку.

**Пример**

*client*

| uid | email | name |
|---|---|---|
| 1 | vlad@gmail.com | vlad |

*admins*

| id | email | name | role |
|---|---|---|---|
| 123 | vlad@gmail.com | vlad | admin |
| 456 | lim@gmail.com | lim | super-admin |

```
WITH emails AS (
  SELECT email FROM client
  UNION
  SELECT email FROM admins
)
SELECT email FROM emails
```

Результат:

| email |
|---|
| vlad@gmail.com |
| vlad@gmail.com |
| lim@gmail.com |

```
WITH emails AS (
  SELECT email FROM client
  UNION
  SELECT email FROM admins
)
SELECT DISTINCT email FROM emails
```

Результат:

| email |
|---|
| vlad@gmail.com |
| lim@gmail.com |

То есть убрались дубликаты vlad@gmail.com.

**Пример**

```
WITH emails AS (
  SELECT email, uid FROM client
  UNION
  SELECT email, uid FROM admins
)
SELECT email, uid FROM emails
```

Результат:

| email | uid |
|---|---|
| vlad@gmail.com | 1 |
| vlad@gmail.com | 123 |
| lim@gmail.com | 456 |

```
WITH emails AS (
  SELECT email, uid FROM client
  UNION
  SELECT email, uid FROM admins
)
SELECT DISTINCT email, uid FROM emails
```

Результат:

| email | uid |
|---|---|
| vlad@gmail.com | 1 |
| vlad@gmail.com | 123 |
| lim@gmail.com | 456 |

Результат такой же, потому что (vlad@gmail.com, 1) != (vlad@gmail.com, 123)

**Пример**

*client*

| uid | email |
|---|---|
| 1 | a@a.com |
| 2 | b@b.com |

*admins*

| id | email |
|---|---|
| 1 | a@a.com |
| 3 | d@d.com |

1.

```
WITH emails AS (
  SELECT email, uid, 'cl' as ty FROM client
  UNION
  SELECT email, uid, 'adm' as ty FROM admins
)
SELECT email, uid, ty FROM emails
```

Результат:

| email | uid | ty |
|---|---|---|
| a@a.com | 1 | cl |
| a@a.com | 1 | ad |
| b@b.com | 2 | cl |
| d@d.com | 3 | ad |

2.

```
WITH emails AS (
  SELECT email, uid, 'cl' as ty FROM client
  UNION
  SELECT email, uid, 'adm' as ty FROM admins
)
SELECT DISTINCT ON (email) email, uid, ty FROM emails
```

Результат:

| email | uid | ty |
|---|---|---|
| a@a.com | 1 | cl |
| b@b.com | 2 | cl |
| d@d.com | 3 | ad |

Отсеяны дубли по email

3.

```
WITH emails AS (
  SELECT email, uid, 'cl' as ty FROM client
  UNION
  SELECT email, uid, 'adm' as ty FROM admins
)
SELECT DISTINCT ON (ty) email, uid, ty FROM emails
```

Результат:

| email | uid | ty |
|---|---|---|
| a@a.com | 1 | cl |
| a@a.com | 1 | ad |

Просто отсеяли все дубли по ty и оставлили первые строки

4.

```
WITH emails AS (
  SELECT email, uid, 'cl' as ty FROM client
  UNION
  SELECT email, uid, 'adm' as ty FROM admins
)
SELECT DISTINCT ON (email, uid) email, uid, ty FROM emails
```

Результат:

| email | uid | ty |
|---|---|---|
| a@a.com | 1 | cl |
| b@b.com | 2 | cl |
| d@d.com | 3 | ad |

Убрался дубль (a@a.com, 1, cl) != (a@a.com, 1, ad)

5.

```
WITH emails AS (
  SELECT email, uid, 'cl' as ty FROM client
  UNION
  SELECT email, uid, 'adm' as ty FROM admins
```

```
)
SELECT DISTINCT ON (email, ty) email, uid, ty FROM emails
```

Результат:

| email | uid | ty |
|---|---|---|
| a@a.com | 1 | cl |
| a@a.com | 1 | ad |
| b@b.com | 2 | cl |
| d@d.com | 3 | ad |

Потому что по этим двум атрибутам нет дубликатов

# Подзапросы

## Как работает `SELECT` в общем случае

1. Сначала вычисляются все элементы, которые идут после `FROM`, то есть таблицы, виртуальные таблицы. Если таблиц >1, то есть `SELECT * FROM client c, admins a`, то формируется их декартово произведение.
2. Если есть `WHERE`, то из декартова произведения убираются строки, не подходящие под условие.
3. Если есть `GROUP BY`, то результрующие строки групируются по указанным атрибутам, затем вычисляются значения агрегатных функций. Если есть `HAVING`, то отфильтровываются группы по условию.
4. Ключевое слово `SELECT` есть всегда, но после него могут идти имена атрибутов, их комбинации, созданные с использованием арифметических или других операций, а также вызовы функций (встроенных или пользовательских). Не все атрибуты обязаны присутствовать после `SELECT`. `SELECT DISTINCT` удаляет дубли.
5. Если есть `ORDER BY`, то строки сортируются по указанным атрибутам, по умолчанию ASC(возрастание).
6. Если есть `LIMIT` или `OFFSET`, то возвращается подмножество строк из выборки.

Это логический уровень. На уровне реализации уже работает планировщик и составляет план выполнения запроса.

**Пример**

```
SELECT * FROM vacancy
WHERE agent_reward >= (SELECT AVG(agent_reward) FROM vacancy)
```

Главным запросом является первый `SELECT`, а то, что в круглых скобках - подзапрос.

Подзапросы пишутся в круглых скобках и могут быть в `SELECT`, `FROM`, `HAVING`, `WITH`.

**Скалярный подзапрос** - это подзапрос, который возвращает скалярное одно значение (1 строку или 1 столбец), с которым можно сравнить другие скалярные значения.

Если подзапрос возвращает набор значений, то его можно использовать в предикате `IN` и `NOT IN`, который позволяет осуществлять проверку наличия элемента в наборе.

**Некоррелируемый подзапрос** - подзапрос, который может быть выполнен сам по себе и не ссылается на колонки основного запроса.

**Пример**

Допустим я хочу получить вакансии, которые в городах с метро:

```
SELECT * FROM vacancy
WHERE city IN (
  SELECT city_name
  FROM city_metro
  WHERE metro_list IS NOT NULL
)
```

Подзапрос можно выполнить отдельно и мы получим просто список городов с метро.

**Коррелируемый подзапрос** - подзапрос, который использует ссылку на столбцы основного запроса.

**Пример**

Допустим я хочу получить список юзеров, для которых есть тикет валидации:

```
SELECT * FROM client c
WHERE EXISTS (
  SELECT 1
  FROM client_validation cv
  WHERE c.uid = cv.uid
);
```

Этот подзапрос выполнится для каждой строки, обрабатываемой во внешнем запросе, но по идее планировщик может сделать так, что мы избежим обработки для всех строк.

Также бывают вложенные подзапросы:

```
SELECT * FROM t1
WHERE EXISTS (
  SELECT 1
  FROM t2
  WHERE param IN (
    SELECT
    -- ...
  )
);
```

## CTE

**CTE (Common Table Expression)** - общее табличное выражение, временная таблица (именованная), которая существует только внутри этого SQL запроса.

CTE используют по смыслу, как подзапрос, то есть создают промежуточный результат, из которого потом используются данные во внешнем запросе.

В отличие от подзапросов, CTE можно использовать несколько раз.

Также CTE можно удобнее использовать для рекурсий.

**Пример**

```
WITH RECURSIVE category_tree AS (
  SELECT id, name, parent_id, 1 as lvl
  UNION ALL
  SELECT c.id, c.name, c.parent_id, ct.lvl + 1
  FROM categories c
  JOIN category_tree ct ON ct.id = c.parent_id
) SELECT *
FROM category_tree
ORDER BY lbl, id;
```

Рекурсивные CTE нужны для деревьев, иерархий, графов, цепочек связей.

**Отличия выполнения**

Если написать

```
WITH res AS MATERIALIZED (
  --...
)
```

То результат будет вычисляем сразу.

Если

```
WITH res AS NOT MATERIALIZED (
  --...
)
```

То `NOT MATERIALIZED` говорит "попытайся встроить в основной запрос" оптимизатору.

**Пример**

Обычная CTE:

```
WITH filtered_promos AS (
  SELECT
    vpc.vacancy_id,
    array_agg(json_build_object('id', pc.id, 'name', pc.name)) AS formatted_promos
  FROM vacancy_promos vpc
  JOIN promos pc
  ON pc.id = vpc.promo_id
  GROUP BY vpc.vacancy_id
) SELECT *
FROM filtered_promos;
```

CTE можно также использовать для `INSERT/UPDATE/DELETE`

**Пример**

```
WITH orders_cnt AS (
  SELECT client_uid,
    COUNT(*) AS cnt
  FROM orders
  GROUP BY client_uid
)
UPDATE client c
SET is_vip = TRUE
FROM orders_cnt oc
WHERE c.uid = oc.client_uid AND oc.cnt > 5;
```

# Изменение данных

## Виртуальные таблицы

Виртуальная таблица - результат, который для SQL выглядит, как таблица.

У него есть строки и столбцы, можно его исполнить в `FROM`, делать к нему `JOIN` и т.д.

Но необязательно он существует, как физическая таблица в БД.

**Пример**

```
SELECT * FROM client -- физическая таблица
```

```
SELECT *
FROM (
  SELECT uid, email
  FROM client
  WHERE deleted_at IS NULL
) as active_clients -- виртуальная таблица, результат подзапроса
```

**VALUES** - создает виртуальную таблицу в запросе.

В основном используется в `INSERT` или чтобы собрать таблицу на ходу.

**Пример**

```
SELECT * FROM (
  VALUES
    (1, 'a@a.com'),
    (2, 'b@b.com'),
    (3, 'c@c.com')
) AS client(uid, email);
```

**Пример**

```
INSERT INTO client (uid, email)
VALUES
  (1, 'a@a.com'),
  (2, 'b@b.com'),
  (3, 'c@c.com');
```

**TEMP** - нужно для создания физической временной таблицы в рамках подключения.

Если создать таблицу с `TEMP`, то она будет как обычная таблица, но будет видна только в рамках одного подключения.

Для ее удаления нужно или прописать действие

```
ON COMMIT DROP/DELETE ROWS
```

или разорвать подключение.

**Пример**

```
CREATE TEMP TABLE tmp AS (
  SELECT * FROM table WITH NO/WITH DATA
) ON COMMIT DROP;
```

# INSERT

При `INSERT` могут возникать проблемы нарушения целостности (ключей), для этого есть `ON CONFLICT`

`ON CONFLICT`:

- `DO NOTHING` - игнорирование ошибки
- `DO UPDATE SET` - **!!!только для той же таблицы!!!** обновить существующую строку (та с которой конфликт)

```
INSERT ... ON CONFLICT [имя_поля] DO ...;
```

`EXCLUDED` - специальная псевдотаблица в `ON CONFLICT DO UPDATE SET`, содержащая строку, которую хотели вставить.

**Пример**

```
INSERT INTO client(uid, email, name)
VALUES ('123', 'a@a.com', 'Bob')
ON CONFLICT (email) DO UPDATE SET name=EXCLUDED.name;
```

Также можно использовать старое значение:

```
CREATE TABLE user_stats(
  user_id UUID PRIMARY KEY,
  visits INTEGER NOT NULL DEFAULT 0
);

INSERT INTO user_stats(user_id, visits)
VALUES ('123', 1)
ON CONFLICT (user_id) DO UPDATE
SET visits=user_stats.visists + EXCLUDED.visits;
```

Можно указать конфликт не по колонкам, а по имени ограничения, для это используют:

```
ON CONFLICT ON CONSTRAINT имя_ограничения DO ...;
```

`ON CONFLICT DO UPDATE` гарантирует атомарность выполнения вставки или обновления, то есть проверка конфликта и последующая вставка/обновление выполняются атомарно. То есть если одна транзакция поймала конфликт, то другая не может между проверкой и вставкой первой этот конфликт убрать.

Для массовой вставки строк используются `COPY`

**Пример**

```
COPY aircrafts_tmp FROM 'path_to_file.txt';
-- или для .csv
COPY aircrafts_tmp FROM 'path_to_file.csv' WITH (FORMAT CSV);
```

Сам файл должен быть такой:

```
IL [табуляция] IlyushinIL96 [табуляция] 9800
\. <- символ для конца файла
```

Или

```
773,Boeing777,11100
...
```

# UPDATE

Тут в принципе ничего нового, работает почти как `INSERT`

`UPDATE` изменяет уже существующие строки таблицы.

Базовый синтаксис:

```
UPDATE table_name
SET column1 = value1,
    column2 = value2
WHERE condition;
```

Если не указать `WHERE` , то обновляется все строки.

Можно делать обновление на основе текущего значения:

```
UPDATE accounts
SET balance = balance + 100
WHERE id = 1;
```

Можно обновлять несколько полей:

```
UPDATE products
SET price = price * 1.1,
    updated_at = NOW()
WHERE id = 1;
```

### UPDATE + FROM

Для `UPDATE` можно использовать `FROM` для использования данных или для условия из другой таблицы.

```
UPDATE users u
SET company_name = c.name
FROM companies c
WHERE u.company_id = c.id;
```

Важно при использовании `FROM` организовать условие, при котором обновляемой строке соответствует только одна строка в другой таблице, иначе может быть некорректное поведение (так как выбирать нужно из некскольких строк).

## DELETE

Тут тоже особо ничего нового.

`DELETE` удаляет строки из таблицы.

Базовый синтаксис:

```
DELETE FROM table_name
WHERE condition;
```

Если не указать `WHERE` , то удаляется все строки.

### Разница между `DELETE` и `TRUNCATE`

`DELETE`

- удаляет строки

- можно с `WHERE`
- построчная операция в логическом смысле
- может быть медленнее на больших объемах
- срабатывают триггеры `DELETE`

`TRUNCATE`

- быстро очищает всю таблицу
- без `WHERE`
- ближе к “мгновенно опустошить таблицу”
- ведет себя иначе с точки зрения блокировок и триггеров

**НИЖЕ ПРО USING + DELETE**

# RETURNING

`RETURNING` позволяет сразу вернуть строки, которые были вставлены, обновлены или удалены.

Это нужно, чтобы после `UPDATE/INSERT/DELETE` не делать дополнительный `SELECT` .

**Пример**

```
UPDATE users
SET name = 'New Name'
WHERE id = 1
RETURNING id, name, updated_at;
```

```
DELETE FROM users
WHERE id = 1
RETURNING id, email;
```

```
UPDATE tickets
SET status = 'approved',
    updated_at = now()
WHERE id = $1
RETURNING id, status, updated_at;
```

Или очень полезный паттерн `RETURNING` в CTE:

```
WITH deleted_rows AS (
    DELETE FROM users
    WHERE is_inactive = true
```

```
    RETURNING id, email
)
SELECT *
FROM deleted_rows;
```

# USING

`USING` можно использовать в нескольких кейсах.

1. `JOIN ... USING имя_поля`
2. `DELETE ... USING имя_другой_таблицы WHERE условие`

### Для JOIN

Если поле, по которому джоиним называется в обеих таблицах одинаково, то можно написать так:

```
SELECT *
FROM users
JOIN companies USING (company_id);

-- сокращение для

SELECT *
FROM users
JOIN companies ON users.company_id = companies.company_id;
```

### Для DELETE

Для `DELETE USING` используется для условия удаления, для которого нужно значение поля другой таблицы. Образно это delete с join логикой.

```
DELETE FROM target_table
USING other_table
WHERE ...;


DELETE FROM t1
USING t2
WHERE t1.some_id = t2.id
  AND t2.some_flag = true;
```

Строки тут удалятся только из t1, t2 нужно для условия.

## TRUNCATE

`TRUNCATE` удаляет все строки из таблицы.

**Пример**

```
TRUNCATE table_name;
```

Используется вместо `DELETE FROM table_name` , так как `DELETE` проходится построчно, а тут все сразу удаляется.

# Индексы

Строки в таблицах хранятся в неупорядоченном виде.

При выполнении операции выборки, обновления и удаления СУБД должна отыскать нужные строки.

Для ускорения этого поиска создается индекс.

**Индекс** - это дополнительная структура данных, которая помогает быстрее находить строки, не просматривая всю таблицу целиком.

Его можно воспринимать как оглавление книги.

Без оглавления пришлось бы листать всю книгу, а с оглавлением я смотрю термиин/главу и тд и перехожу в нужное место.

**В общем упращенно индекс хранит значение индексируемого поля и ссылку на строку в таблице (ссылку на tuple/heap location)**

```
email='c@example.com' -> строка №...
```

Но вообще это не просто список, у разных типов индексов разная внутренняя структура:

- B-Tree - дерево
- Hash - хэш таблица / бакеты
- GIN - inverted structure
- BRIN - summary по блокам

Индексы хранят данные в отсортированном виде, что помогает быстрее находить данные.

При выполнении поиска конкретных строк в таблице специальная подсистема СУБД, называемая **планировщиком**, проверяет, имеется ли для этой таблице индекс, созданный на основе тех же столбцов, что указаны, например, в `WHERE`.

Если он есть и его использование целесообразно, то сначала выполняется поиск по индексу, а затем, если такие значения в нем найдены, производится обращение к таблице с использованием указателей, которые хранятся в записях индекса.

Таким образом полный перебор может быть заменен поиском в упорядоченном индексе и переходом к строке в таблице по прямому указателю.

Индексы нужно создавать, опираясь на наиболее часто выполняемые запросы/выборки.

**Зачем нужны**

Индексы полезны для:

- `WHERE`
- `JOIN`, если создан индекс по соединяемым полям
- `GROUP BY` в некоторых сценариях
- `ORDER BY` если есть индекс по сортируемому полю
- проверка уникальности
- поиск по диапазону
- полнотекстовый поиск
- JSONB/array/search-операторы - через специальные типы индексов

## Цена индексов

Индексы являются неким компромиссом:

- быстрее читаем
- дороже пишем

Основные плюсы:

- ускоряет чтение
- ускоряет поиск по условиям
- может ускорять join
- может обеспечивать уникальность

Основные минусы:

- занимает место на диске
- замедляет INSERT

- замедляет UPDATE
- замедляет DELETE
- требует обслуживания
- может раздуваться
- не каждый запрос умеет использовать индекс эффективно

**Индекс эффективен**, когда запрос выбирает малую долю таблицы.

То есть если у меня около 1% строк в таблице селектятся, то выгоднее быстро найти нужные ключи в индексе, затем быстро достать эти несколько строк из таблицы.

**Индекс неэффективен**, когда запрос выбирает большую долю таблицы.

Например:

```
SELECT * FROM users WHERE is_active = true;
```

Если `is_active=true` у 95% строк, то дешевле сделать Seq Scan по таблице и достать все строки нежели пойти в индекс, получить почти все ссылки на строки и по этим ссылкам прочитать всю таблицу.

**Селективность** - насколько условие отфильтровывает данные.

*Высокая селективность*

```
WHERE email = 'unique@example.com';
```

Нашли 1 строку из всех пользователей.

*Низкая селективность*

```
WHERE is_active = true;
```

Индекс еще зависит от:

- ширины строк
- того, нужны ли все столбцы или только часть
- есть ли index-only scan
- насколько данные лежат близко на диске
- оценки planner’a

Индексы нужно создавать на поля с высокой селективностью.

## Обновление индекса

При `INSERT/UPDATE/DELETE` индекс обновляется инкрементально.

При `INSERT`:

- строка пишется в таблицу
- для каждого индекса добавляется новая запись в соответствующую структуру

При `UPDATE`:

Если изменилось индексируемое поле, то старый ключ больше не актуален, новый ключ добавляется в индекс.

Если индексируемое поле не изменилось, то будет HOT UPDATE.

При `DELETE`:

- строка удаляется из таблицы
- соответствующая индексная запись тоже становится неактуальной
- потом vacuum участвует в очистке мусора

## B-Tree

B-Tree - сбалансированное многоветвистое дерево поиска.

Основные свойства:

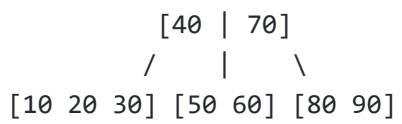
- не бинарное, узел может содержать много ключей
- дерево остается сбалансированным
- все leaf-узлы примерно на одной глубине
- глубина дерева маленькая даже при большом объеме данных

**Пример B-Tree**

Допустим, у нас есть значения:

10, 20, 30, 40, 50, 60, 70, 80, 90

Упрощенно дерево может выглядеть так:




Как это читать:

корень содержит ключи-разделители 40 и 70

если ищем:

меньше 40 → идем в левый ребенок

от 40 до 70 → в средний

больше 70 → в правый

**Поиск в B-Tree**

Ищем 60.

Смотрим корень:

60 > 40

60 < 70

значит идем в средний узел

В среднем узле лежит:

[50 60]

находим 60

Если бы это был индекс PostgreSQL, в leaf-узле рядом еще лежала бы ссылка на строку таблицы.

Это удобно, так как даже если в таблице миллионы строк, глубина дерева может быть 3-5 уровней.

B-Tree подходит также для диапазонов, потому что:

1. Быстро находит первое значение диапазона
2. Потом идет по соседним leaf-страницам
3. Собирает все подходящие записи подряд

Из-за этого хорош и для `=`, и для `< > BETWEEN`, и для `ORDER BY`

## Составной индекс

Составной индекс - индекс по нескольким полям.

При создании важен порядок полей:

```
CREATE INDEX idx_orders_user_status_created
ON orders(user_id, status, created_at);
```

Здесь порядок:

1. user_id
2. status
3. created_at

Означает, что внутри индекса сортировка идет в порядке:

- сначала по `user_id`
- внутри одинаковых `user_id` по `status`
- внутри одинаковых `(user_id, status)` по `created_at`

Это будет влиять на то, какие запросы будут эффективно использовать индекс.

Например, для кейса сверху будут хорошо работать запросы по:

```
WHERE user_id = ...
WHERE user_id = ... AND status = ...
WHERE user_id = ... AND status = ... AND created_at > ...
```

При выборе порядка важно смотреть на типичные запросы, первым лучше ставить поле, на которое больше уникальных значений.

Лучше первым ставить уникальное поле, которое сравниваем через `=`.

**В общем случае порядок такой**

1. поля, по которым почти всегда идет точное равенство `=`
2. потом поля для диапазона
3. потом поля для сортировки, если это помогает паттерну запроса

Но это просто частая практика, а не правило.

## Определение полей для индексации

Часто нужно смотреть на паттерны запросов. В основном нужны на поля, которые часто участвуют в:

- `WHERE`
- `GROUP BY` иногда
- `ORDER BY`

- `JOIN ON`
- `UNIQUE`
- foreign key-side points

**Хорошие кандидаты:**

Поля для поиска одной/малого числа строк

- `email`
- `phone`
- `external_id`
- `uuid`

Поля для join

- `user_id`
- `company_id`
- `vacancy_id`

Поля для частых диапазонов / сортировок

- `created_at`
- `published_at`

Комбинации полей из типовых запросов

- `(user_id, created_at)`
- `(company_id, status)`
- `(status, published_at)` — если реально так ищут

**Плохие кандидаты:**

- булевы поля без редкого распределения
- поля, по которым почти нет фильтрации
- столбцы, которые почти никогда не участвуют в запросах
- поля, которые часто меняются, если индекс не дает ощутимой пользы

**Основная мысль**

Индекс часто нужен не под конкретную колонку в принципе, а под конкретный частый или важный запрос.

Если много записи/удаления и много индексов, то будет дорого.

# Оптимизация запросов

**Метод доступа** - характеризует способ, который используется для просмотра таблиц и извлечения только тех строк, которые соответсвуют критерию отбора.

Методы доступа:

- Последовательный просмотр (Sequential Scan)
- Индексный просмотр (Index Scan)
- Просмотр только по индексу (Index Only Scan)
- Просмотр битовой карты (Bitmap Scan)

Так как таблицы, индексы хранятся на диске, то для работы с ними эти объекты считываются в память, в которой разбиваются на отдельные фрагменты (страницы).

Страницы имеют специальную структуру и по умолчанию размер равен 8кБайт.

## Planner

**Planner** - подсистема СУБД, которая выбирает, как выполнять запрос и строит его план.

Planner решает:

- как читать таблицы
- использовать ли индекс
- как соединять таблицы
- в каком порядке соединять таблицы
- где применять фильтр
- где сортировать
- где агрегировать

Потом **executor** берет этот план и выполняет его.

**План** - порядок действий, которые выполнятся при выполнении запроса.

Planner строит план как дерево.

**Самый верхний узел** - это последняя операция, которая выдает результат ( `LIMIT` ли другие).

Далее идут узлы с **JOIN** таблиц, в них описано какой подход ( `Nested Loop` , `Hash Join` , `Merge Join` и т.д.) выбран для соединения таблиц.

Далее идут узлы с **сканированием таблиц**, в них описано как мы читали данные из таблиц ( `Index Scan` , `Bitmap Index Scan` , `Seq Scan` и т.д.).

Запрос по плану выполняется **снизу вверх**.

`EXPLAIN` - показать план запроса.

`EXPLAIN ANALYZE` - показать план запроса, выполнить его и показать фактические результаты выполнения.

## Сканирование таблиц

**Seq scan (sequential scan)** - обращение к индексам не происходит, строки извлекаются из heap (табличных страниц) в соответствии с условием.

Например, когда мы не указываем `WHERE` , то будут вычитаны все строки из heap.

Или же если мы выбираем большую часть строк таблицы, то есть селективность выборки низкая и индекс не ускоряет поиск, а замедляет.

**Пример**

```
SELECT *
FROM users
WHERE age > 30;
```

Без подгодящего индекса PostgreSQL может сделать:

```
Seq Scan on users
  Filter: (age > 30)
```

Это значит, что:

- PostgreSQL читает все строки heap подряд
- для каждой строки проверяет условие `age > 30`

Это нормально, когда:

- таблица маленькая
- условие возвращает большую часть таблицы
- индекс отсутствует
- индекс есть, но planner считает, что полностью просканить дешевле

**Index scan** - просмотр на основе индекса, предполагает, что мы обращаемся к индексу таблицы. После отыскания нужных ссылок на строки в таблицах производится обращение к соответствующей странице таблицы и извлечение строки. При этом нужно учитывать, что несмотря на то, что в индексах данные упорядочены, но физически они могут лежать в разных страницах и обращение к страницам происходит хаотично.

Таким образом если селективность выборки низкая, индекс может замедлить поиск.

Если коротко, то:

1. идем индекс
2. находим подходящие ключи
3. по ссылкам идем в heap
4. читаем реальные строки

**Пример**

```
SELECT *
FROM users
WHERE email = 'a@example.com';
```

План:

```
Index Scan using idx_users_email on users
  Index Cond: (email = 'a@example.com')
```

Это значит, что:

- индекс помог найти нужную строку
- потом PostgreSQL пошел в heap за самой строкой

**Index only scan** - просмотр только на основе индекса. Эффективен, когда получаемые данные редко изменяются.

Сначала выполняется обращение к карте видимости (visibility map), в которой одним битом отмечены страницы, на которых содержатся строки, видимые всем транзакциям. Если версия лежит на такой странице, то обращение к таблице не нужно.

Но если строки лежат на другой странице, то все равно будет обращение к таблице.

Он в основном используется, когда в `SELECT` столбцы, которые все имеют индекс.

**Пример**

- есть индекс по `(email, created_at)`
- запрос просит только `email` и `created_at`

```sql
SELECT email, created_at
FROM users
WHERE email = 'a@example.com';
```

Если все гуд, то PostgreSQL может взять данные прямо из индекса и не вычитывать из heap, поскольку это дороже.

Это не всегда работает, так как по MVCC у нас не всегда строки могут быть видимы всем транзакциям.

**Bitmap scan** - просмотр на основе битовой карты.

Это модификация просмотра на основе индекса. Он позволяет оптимизировать индексный поиск за счет того, что сначала ищет строки в индексе и формирует битовую карту, в которой указано, на каких страницах лежат эти строки.

После того, как карта сформирована, выполняется извлечение строк из страниц таблицы, но при этом обращение к каждой странице производится только один раз.

То есть читаем heap пачками.

Если коротко, то bitmap - компромисс между index scan и seq scan.

Образно он сначала собирает список "на каких страницах и в каких местах лежат подходящие строки", затем ходит в heap пачками.

Bitmap подход осуществляется в два шага:

1. `Bitmap Index Scan` - PostgreSQL идет по индексу, собирает набор ссылок на подходящие строки в виде bitmap/карты
2. `Bitmap Heap Scan` - PostgreSQL идет по карте и читает пачками строки из heap

## Соединение таблиц

При составлении плана planner выбирает, как оптимальнее соединить таблицы.

Чаще всего используются:

- **Nested Loop**
- **Hash Join**
- **Merge Join**

**outer/inner** - внешняя/внутренняя таблица. Внешняя - та по которой Nested Loop, внутренняя, по которой ищутся совпадения.

Например:

```
Nested Loop
  -> Index Scan on users u
       Index Cond: (id < 10)
  -> Seq Scan on orders o
       Filter: (o.user_id = u.id)
```

Тут:

- outer - users
- inner - orders

Это никак не связано с порядком присоединения таблиц при написании самого запроса, planner сам смотрит, какую таблицу выгоднее поставить в качестве inner и outer.

**Nested Loop** - для каждой строки внешней таблицы выполняется поиск совпадений во внутренней таблице.

Этот поиск может осуществляться:

- полным проходом (Seq scan)
- индексным проходом (Index scan)
- чем-то еще...

Хорош, когда:

- внешняя сторона маленькая (таблица в цикле маленькая)
- когда для внутренней стороны есть быстрый поиск (часто индекс)

Плох, когда:

- внешняя сторона большая
- внутренняя сторона каждый раз ищется долго

**Hash Join** - для одной таблицы (стороны) строится hash table, ключом которой является колонка, по которой мы джоиним, далее по другой таблице проходятся и ищут совпадения по ключу.

Хорош, когда:

- join по `=`
- данных уже много
- nested loop дорогой

**Merge Join** - работает, когда обе стороны отсортированы по ключу join.

Можно воспринять, как проход по двум отсортированным спискам двумя указателямию

```
users.id:   1 3 5 8
orders.uid: 1 1 3 5 5 9
```

Хорош, когда:

- данные уже отсортированы или сортировка оправдана
- join по `=`

## Cond

`Cond` - condition, условие.

Разные `Cond` в плане значат разные места применения условия.

### `Index Cond`

Условие, которое используется прямо на уровне индекса

Например:

```
Index Scan using idx_users_email on users
  Index Cond: (email = 'a@example.com')
```

Это значит, что индекс реально используется по условию:

```
email = 'a@example.com'
```

### `Filter`

Условие, которое проверяется после чтения строк

Например:

```
Index Scan using idx_users_company_id on users
  Index Cond: (company_id = 10)
  Filter: (status = 'active')
```

Это значит:

- индекс помог найти всех пользователей компании 10
- потом уже среди них PosgreSQL отфильтровал только `status = 'active'`

Нужно смотреть, чтобы условие было не в `Filter` при использовании index, а в `Index Cond`, потому что иначе индекс неподходящий для запроса.

### `Hash Cond`

Условие, по которому делается `hash join`

Например:

```
Hash Join
  Hash Cond: (o.user_id = u.id)
```

Это значит, что строки соединяются по условию:

```
o.user_id = u.id
```

### `Merge Cond`

Условие, по которому делается `merge join`

Например:

```
Merge Join
  Merge Cond: (o.user_id = u.id)
```

Это значит, что строки соединяются по условию:

```
o.user_id = u.id
```

### `Join Filter`

Это дополнительное условие join, которое проверяется после основной логики join.

### `Recheck Cond`

Часто встречается у bitmap/GIN/GiST

Это значит:

- индекс дал кандидатов
- потом условие нужно перепроверить на уровне строк

# Поля плана

При составлении плана мы сможем видеть стоимость запроса, количество ожидаемых строк, время и другие поля, по которым сможем оценить эффективность запроса.

**`cost=...`** - оценочная стоимость, которую planner использует для альтернативных планнов. Это условная внутренняя метрика. Planner выбирает план с меньшей ожидаемой стоимостью.

**Пример**

```
(cost=0.29..8.31 rows=1 width=64)
```

Здесь:

- `0.29` - startup cost
- `8.31` - total cost

*Startup cost* - это цена на выдачу первых строк.

*Total cost* - это цена на выдачу всех строк.

Это важно, например, для `LIMIT`, потому что иногда план с большим total cost, но маленьким startup cost лучше для запроса, которому нужно быстро отдать первые строки.

**`rows=...`** - количество строк, которые planner ожидает получить из узла.

Если эта ошибка сильно ошибкается, то planner может выбрать плохой план.

По идее если у нас большая разнца в ожидаемых rows и фактически полученных, то план кривой.

**`with=...`** - оценка среднего размера строки в байтах. Planner использует это в своих расчетах стоимости.

**`actual time=...`** - появляется в плане при использовании `EXPLAIN ANALYZE`.

Показывает фактическое время работы узла.

**`actual rows=...`** - сколько строк реально вернул узел.

**`loops=...`** - сколько раз этот узел был выполнен.

Это особенно важно для nested loop: внутри внешнего цикла внутренний узел может быть повторен очень много раз.

**`val1..val2`** - от val1 до val2 включительно.

# Разбор плана

### Пример

```
EXPLAIN ANALYZE
SELECT v.id, v.company_id, c.name
FROM vacancy v
JOIN company c ON v.company_id = c.id
WHERE v.cv_required = false
LIMIT 100
```

План запроса:

```
Limit  (cost=6.10..45.87 rows=100 width=46) (actual time=0.076..0.187 rows=100 loops=1)
  ->  Hash Join  (cost=6.10..91.60 rows=215 width=46) (actual time=0.075..0.179 rows=100 loops
        Hash Cond: (v.company_id = c.id)
        ->  Seq Scan on vacancy v  (cost=0.00..84.89 rows=215 width=32) (actual time=0.010..0.
              Filter: (NOT cv_required)
              Rows Removed by Filter: 13
        ->  Hash  (cost=5.49..5.49 rows=49 width=30) (actual time=0.047..0.048 rows=53 loops=1
              Buckets: 1024  Batches: 1  Memory Usage: 12kB
              ->  Seq Scan on company c  (cost=0.00..5.49 rows=49 width=30) (actual time=0.006
Planning Time: 1.405 ms
Execution Time: 0.294 ms
```

Разбор плана:

### Шаг 1

Сначала смотрим на нижние узлы, узел 1 у нас:

```
->  Seq Scan on company c  (cost=0.00..5.49 rows=49 width=30) (actual time=0.006..0.030 rows=5
```

Тут у нас читается вся таблица `company`

Реально было возвращено 53 строки, первую отдели через 0.006s, все строки отдали через 0.030s.

Узел был выполнен `loop=1` раз.

### Шаг 2

Далее смотрим на узел 2 `Hash`

```
->  Hash  (cost=5.49..5.49 rows=49 width=30) (actual time=0.047..0.048 rows=53 loops=1)
      Buckets: 1024  Batches: 1  Memory Usage: 12kB
```

Тут у нас строится hash table по результату `company`

Дополнительные поля:

- `Buckets: 1024` - сколько бакетов в hash table
- `Batches: 1` - hash целиком уместился в памяти, без разбиения на батчи
- `Memory Usage: 12kB` - количество памяти, которое нужно для хранения hash table

**Шаг 3**

Далее смотрим на узел 3, это `Seq Scan on vacancy v`

```
->  Seq Scan on vacancy v  (cost=0.00..84.89 rows=215 width=32) (actual time=0.010..0.082 rows
      Filter: (NOT cv_required)
      Rows Removed by Filter: 13
```

Тут у нас читается вся таблица `vacancy` один раз

Полученные строки фильтруются по условию `NOT cv_required`

После вычитывания строк до остановки 13 строк не прошли фильтр.

**Шаг 4**

Следующий узел 4 это `Hash Join`

```
->  Hash Join  (cost=6.10..91.60 rows=215 width=46) (actual time=0.075..0.179 rows=100 loops=1
      Hash Cond: (v.company_id = c.id)
```

Здесь у нас:

- взялся поток `vacancy` после фильтрации
- для каждой vacancy в заранее построенный в узле 2 hash table ищется компания

`Hash Cond` - условие соединения:

```
v.company_id = c.id
```

**Шаг 5**

Узел 5: `LIMIT`

```
Limit  (cost=6.10..45.87 rows=100 width=46) (actual time=0.076..0.187 rows=100 loops=1)
```

Тут у нас вовзращаются первые 100 строк, подошедшие по условию.

Причем по факту было вычитано 113 строк, так как 13 из них отбросились фильтром.

### Пример

```
EXPLAIN ANALYZE
SELECT v.id, v.company_id, c.name
FROM vacancy v
JOIN company c ON v.company_id = c.id
LIMIT 100
```

План запроса:

```
Limit  (cost=0.15..26.40 rows=100 width=46) (actual time=0.025..0.193 rows=100 loops=1)
  ->  Nested Loop  (cost=0.15..102.28 rows=389 width=46) (actual time=0.024..0.185 rows=100 lo
        ->  Seq Scan on company c  (cost=0.00..5.49 rows=49 width=30) (actual time=0.009..0.01
        ->  Index Scan using idx_vacancy_company_id on vacancy v  (cost=0.15..1.91 rows=7 widt
              Index Cond: (company_id = c.id)
Planning Time: 1.680 ms
Execution Time: 0.331 ms
```

Разбор плана:

### Шаг 1

Смотрю на нижние узлы, узел 1 у нас `Index Scan on vacancy v`

```
->  Index Scan using idx_vacancy_company_id on vacancy v  (cost=0.15..1.91 rows=7 width=32) (a
      Index Cond: (company_id = c.id)
```

Тут у нас читается таблица `vacancy` с использованием индекса по `company_id`

При этом `Index Cond` говорит о том, что индекс для фильтрации использовался.

Было вычитано в среднем по 4 строки из `vacancy` для каждой компании. Точнее за один запуск.

Этот узел был выполнен `loop=27` раз, то есть скорее всего у нас 27 компаний вернулось в итоге.

То есть около 27*4 ~= 108 строк кандидатов на отдачу было вычитано.

Из-за лимита в результате наружу ушло ровно 100 строк.

**Шаг 2**

Следующий узел 2 это `Seq Scan on company c`

```
->  Seq Scan on company c  (cost=0.00..5.49 rows=49 width=30) (actual time=0.009..0.018 rows=2
```

Тут у нас вычиталась один раз последовательно таблица `company` до исчерпания `LIMIT`

Как раз вычиталось 27 строк.

**Шаг 3**

Следующий узел 3 это `Nested Loop`

```
->  Nested Loop  (cost=0.15..102.28 rows=389 width=46) (actual time=0.024..0.185 rows=100 loop
```

Тут мы для строк из потока `company` ищем подходящие строки в таблице `vacancy` проходом с использованном в узле 1 индексом.

**Шаг 4**

Узел 4: `LIMIT`

```
Limit  (cost=0.15..26.40 rows=100 width=46) (actual time=0.025..0.193 rows=100 loops=1)
```

Тут мы вернули первые 100 строк, которые подошли по условию.

## Время выполнения

Общее время от "отправил запрос" до "получил ответ" = `planning time + execution time`

В PostgreSQL:

`Planning time` - время потраченное на:

- парсинг запроса
- переписывание запроса
- работу planner
- выбор плана

`Execution time` - время потраченное на:

- реальное выполнение запроса
- получение результата

По идее на маленьких запросах planning time может быть намного больше, чем execution time.

Это нормально.

Но на тяжелых запросах важнее всего смотреть на execution time.

## Дополнительные узлы

**`Materialize`** - узел, который сохраняет результат дочернего узла в памяти/временном буфере, чтобы не вычислять его заново каждый раз.

Можно воспринимать как кеш в рамках текущего запроса в рамках текущего execution plan, а не глобальный кеш базы.

**`Memoize`** - узел, который кеширует результаты поиска по ключу, чтобы не выполнять узел еще раз, если уже значение по ключу было найдено

**`Cache Key`** - ключ, по которому `MEMOIZE` хранит результат

**`Hit/Misses`** - статистика `Memoize`:

- hit - нашли в кеше
- miss - пришлось заново идти искать

**`Evictions`** - сколько записей было вытеснено из `MEMOIZE` -кэша из-за того, что кэш начал не влезать в выделенную память

**`Memory Usage`** - сколько памяти занял кэш этого узла

# Storage

При создании таблиц они физически хранятся на диске.

Таблица разбивается на страницы (pages).

При изменении строк PostgreSQL работает с копиями pages в буферах, а не напрямую с page в памяти.

Dirty pages - страница в памяти, которая была изменена, но еще не записана обратно на диск.

В страницах лежат версии строк (tuples).

Набор страниц таблицы образует heap.

Индексы хранятся отдельно и ссылаются на строки в heap.

```
CREATE TABLE users (
    id bigint,
    email text
);
```

```
table users
  -> page 1
      -> tuple
      -> tuple
  -> page 2
      -> tuple
      -> tuple
```

### Page

`Page` - это блок фиксированного размера, в котором PostgreSQL хранит данные.

Обычно размер страницы `8KB`

### Tuple

`Tuple` - это физическая запись строки внутри страницы.

Ввиду MVCC у одной логической строки может быть несколько tuples.

### Heap

`Heap` в PostgreSQL - это основное хранилище строк таблицы.

Когда говорят:

- heap tuple
- heap fetch
- heap scan

имеют ввиду чтение/хранение строк в основной таблице.

То есть `heap` хранит реальные физические строки таблицы.

Индексы хранят как раз ссылки на строки в heap.

То есть:

```
users heap
  page 1:
    tuple A
```

```
    tuple B
  page 2:
    tuple C
    tuple D


email index
  "a@mail.com" -> ссылка на tuple A
  "b@mail.com" -> ссылка на tuple C
```

**Физическое хранение таблицы**

`CTID` - физический адрес tuple внутри таблицы

Он хранит `(page 5, tuple 3)` условно

Например:

```
SELECT ctid, * FROM client LIMIT 100
```

Вывод:

```
ctid | uid | email | name
-----+-----+-------+------
(0,1) | 1   | a@a.com | Bob
(2,10) | 2   | b@b.com | Bob
(1,3) | 3   | c@c.com | Bob
...
```

То есть по факту индексы хранят `ctid` .

**Heap fetch**

Это когда PostgreSQL после работы с индексом идет в heap за самой строкой.

- Index Scan = не только индекс, но и поход в heap
- Index Only Scan старается избежать heap fetch
- Heap Fetches в плане показывают, сколько раз все же пришлось идти в heap

**Вставка, MVCC, удаление**

При вставке нового tuple PostgreSQL смотрит в Free Space Map, потом вставляет tuple в heap там, где есть место, только потом индекс начинает ссылаться на новый tuple.

MVCC работает не только с логическими версиями строк, а как раз с tuple, потому что это буквально физически версии строк.

**Free Space Map**

Хранит информацию, где в таблице есть место для новых tuple.

**Visibility Map**

Хранит информацию о страницах, которые "достаточно чистые" с точки зрения видимости, это важно для vacuum и index only scan.

# Vacuum

`VACUUM` - механизм, который чистит мусор, появляющийся из-за MVCC.

Так как:

- при удалении строки физически остаются на диске, просто помечаются как удаленные
- при обновлении/изменении строки физически создаются новые версии строк

То в heap начинают копиться старые версии строк, dead tuples, неиспользуемое место.

**Что делает VACUUM**

1. Находит dead tuples (смотрит, какие версии строк никому не нужны)
2. Освобождает пространство для повторного использования
3. Чистит связанные индексные записи (так как индексы тоже перестраиваются и хранят разные версии)

VACUUM физически не освобождает место, он делает его reusable, поэтому размер файла может не уменьшиться.

По шагам:

1. идет по страницам heap
2. смотрит tuples
3. определяет:
    - живая
    - мертвая
    - еще потенциально видимая
4. dead tuples помечает как освобожденные
5. обновляет служебные структуры
6. очищает индексы от мусора, связанного с этими tuples

**AUTOVACUUM**

`AUTOVACUUM` - это постоянный автоматический запусть VACUUM.

**Что делает autovacuum**

- автоматически запускает vacuum на таблицах
- автоматически запускает analyze
- следит, чтобы мусор не копился слишком сильно
- защищает от xid wraparound

# WAL

`WAL` - Write Ahead Log - журнал предзаписи - подсистема/механизм PostgreSQL.

Прежде чем PostgreSQL посчитает изменение надежным, он сначала записывает информацию об этом изменении в специальный журнал.

Основная цель WAL - это durability из ACID.

То есть если я делаю:

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
COMMIT;
```

И сразу после `COMMIT` сервер падает, то после перезапуска PostgreSQL должен уметь восстановить изменения из WAL.

То есть все изменения сначала пишутся в журнал, затем изменяются таблицы/индексные страницы на диске.

Если грубо, то WAL хранит:

- какая страница была изменена
- какой tuple был вставлен
- какой tuple был обновлен
- какой tuple был удален
- какой split произошел в индексе
- какие служебные изменения были сделаны

**Пример**

Пусть делаем:

```sql
UPDATE users SET name = 'Bob' WHERE id = 1;
```

**Шаг 1**

PostgrSQL находит нужную строку по плану.

**Шаг 2**

Меняет данные в памяти (не на диске, а страница в shared buffers/буферах)

Если это UPDATE, то с точки зрения PostgreSQL обычно:

- создается новая версия tuple
- старая версия остается как старая MVCC-версия

То есть изменяются:

- heap page
- возможно, индексные страницы
- служебные данные

**Шаг 3**

Формируется WAL record, которая описывает изменения, если грубо:

- что изменилось
- где изменилось
- какой тип операции был

**Шаг 4**

WAL record помещается в WAL buffer.

**Шаг 5**

При `COMMIT` PostgreSQL не обязан записать сразу все страницы/индексы на диск, но обязан обеспечить, что WAL-записи достаточные для восстановления этих изменений.

То есть flush WAL на диск и после этого только разрешается `COMMIT` .

**WAL как механизм**

Это целая подсистема PostgreSQL, которая включает:

- генерацию WAL-записей
- буферизацию WAL
- flush WAL на диск

- replay WAL при recovery
- использование WAL для репликации

**SUMMARY**

Кароче WAL - это физически write-ahead log журнал.

Если как подсистема, то это механизм следующий:

При любом изменении сначала пишется изменение в WAL buffers.

Сами измененные страницы также хранятся в buffers, а не сразу физически меняются.

Изменение считается надежным (то есть если все упадет, мы сможем его воспроизвести), когда WAL flushed, то есть из WAL buffers журнал пишется на диск.

Потому что если сервер упал, а на диске еще нет WAL, он был просто во временном WAL buffer, то мы ничего не восстановим.

Есть checkpoint, это состояние базы, до которого все изменеия можно не производить заново из WAL (то есть его можно не replay сначала).

Писанием, читанием из WAL занимаются механизмы типа WAL writer и тд.

## Checkpoint

`CHECKPOINT` - это момент, когда PostgreSQL:

- сбрасывает много dirty pages на диск
- фиксирует точку, с которой потом удобно начинать recovery

То есть до текущего checkpoint изменения уже отражены в основных файлах достаточно, чтобы после падения восстановить, с которой можно не читать WAL сначала.

# Locks

Блокировки - механизм PostgreSQL, который позволяет обеспечить целостность данных при одновременной работе с ними.

Lock - это право или ограничение на выполнение операции с данными, которое один процесс (транзакция) накладывает на объект, чтобы другие процессы не могли изменить или даже прочитать эти данные до завершения операции.

Основная цель бликировок - обеспечить соблюдение принципов ACID.

# Уровни блокировок

Блокировки существуют на нескольких логических и физических уровнях

## Блокировки на уровне объектов (таблиц)

Эти блокировки накладываются командами DDL ( `CREATE` , `ALTER` , `DROP` ) и явными командами `LOCK` .

PostgreSQL старается использовать наименее слабую блокировку, подходящую для команды.

Основные типы (от слабых к сильным):

- `Access ShareLock` : Самая слабая. Накладывается командой `SELECT` на все таблицы в запросе. Совместима со всеми, кроме `ACCESS EXCLUSIVE` .
- `Row ShareLock` : Накладывается `SELECT FOR UPDATE/FOR SHARE` . Говорит: «Я планирую изменять отдельные строки».
- `Row ExclusiveLock` : Накладывается командами `UPDATE, DELETE, INSERT` . Совместима сама с собой, но конфликтует с более сильными блокировками (например, с `SHARE` и `ACCESS EXCLUSIVE` ). Несколько транзакций могут одновременно обновлять разные строки одной таблицы.
- `ShareLock` : Накладывается `CREATE INDEX` (без `CONCURRENTLY` ). Запрещает изменение данных ( `Row Exclusive` ), но разрешает чтение ( `Access Share` ).
- `Share Row ExclusiveLock` : Запрещает и изменение данных, и наложение `ShareLock` . Накладывается `CREATE TRIGGER` и некоторыми другими командами.
- `Access ExclusiveLock` : Самая сильная. Блокирует всё, включая `SELECT` . Накладывается `ALTER TABLE, DROP TABLE, TRUNCATE, VACUUM FULL` .

## Блокировки на уровне строк (Row-Level Locks)

Блокировки на строки не мешают обычному `SELECT` (чтению).

**`FOR UPDATE`** : самая сильная блокировка строки.

Транзакция блокирует строки для своего будущего обновления. Другая транзакция, которая попытается заблокировать эти же строки ( `SELECT FOR UPDATE/FOR SHARE` или выполнить `UPDATE/DELETE` ), будет ждать завершения первой.

```
BEGIN;
SELECT * FROM accounts WHERE id = 1 FOR UPDATE; -- блокирует строку
--Транзакция висит открытой...
```

```
UPDATE accounts SET balance = balance + 100 WHERE id = 1; -- обновляет строку
COMMIT;
```

**FOR NO KEY UPDATE** : похожа на `FOR UPDATE` , но слабее.

Она говорит, что "Я буду менять неключевые поля".

Она не конфликтует с `FOR SHARE` .

**FOR SHARE** : блокировка на чтение с намерением "я не буду менять эту строку, но хочу чтобы никто другой ее не менял, пока я читаю".

Другие транзакции могут накладывать `FOR SHARE` , но `FOR UPDATE` будет ждать.

**KEY SHARE** : самая слабая блокировка. "Я смотрю на первичный ключ, не удаляйте строку". Используется для поддержки внешних ключей.

### Блокировки на уровне предикатов (Predicate Locks)

Используются на уровне изоляции `Serializable` для предотвращения фантомного чтения. Блокируется не конкретная строка, а условие ( `WHERE amount > 1000` ). Это сложный механизм, основанный на блокировках в индексах и таблицах, который гарантирует, что никакая другая транзакция не сможет вставить или изменить данные, подпадающие под это условие.

### Блокировки в памяти (Lightweight Locks/Latches)

Это внутренние, низкоуровневые блокировки, защищающие структуры данных в разделяемой памяти (буферный кеш, хеш-таблицы). Они очень кратковременны и не предназначены для управления пользовательскими данными. Разработчики приложений с ними напрямую не сталкиваются.

## Оптимистичные и пессимистичные блокировки

Это стратегии управления параллельным доступом.

### Пессимистичная блокировка

#### Принцип

"Я предполагаю, что конфликт произойдет с высокой вероятностью, поэтому блокирую данные заранее и не даю никому с ними работать".

#### Реализация

Явное использование `SELECT ... FOR UPDATE`

### Как работает

Прежде чем дать пользователю возможность редактировать данные, транзакция блокирует нужные строки. Пока форма открыта у пользователя, никто другой не может изменить эти данные.

### Пример (кошелек, снятие денег)

```
BEGIN;
-- Пессимистично блокируем строку кошелька
SELECT balance FROM wallets WHERE user_id = 1 FOR UPDATE;
-- Приложение проверяет, что balance >= сумма_снятия
UPDATE wallets SET balance = balance - 100 WHERE user_id = 1;
COMMIT;
```

### Плюсы

Простота и надежность. Конфликт невозможен, как только блокировка получена. Идеально подходит для ситуаций с высокой конкуренцией (запись в очередь, изменение баланса, бронирование билета).

### Минусы

Снижение параллелизма. Если пользователь, заблокировав строку, «ушел пить кофе», все остальные будут ждать. Риск deadlock'ов (взаимоблокировок).

## Оптимистичная блокировка

### Принцип

"Я предполагаю, что конфликт маловероятен. Я разрешаю всем читать и работать с данными, но при сохранении проверю, не изменил ли их кто-то другой".

### Реализация

Не использует встроенные блокировки для ожидания. Основана на версионировании (добавлении колонки `version` или использовании системной `xmin`).

### Как работает

Клиент читает данные и номер версии (например, `version = 5`). Пока он редактирует, данные не заблокированы. При сохранении он выполняет `UPDATE` с условием `WHERE version = 5 AND ...`, и заодно сам увеличивает версию до 6. Если `UPDATE` затронул 0 строк, значит, версия уже не 5 (кто-то успел изменить данные раньше). Приложение получает ошибку и предлагает пользователю обновить данные и попробовать снова.

### Пример (редактирование профиля пользователя)

```
-- Шаг 1: Читаем данные
SELECT username, email, version FROM user_profiles WHERE user_id = 1;
-- Допустим, вернулось: ('Alice', 'alice@mail.com', 5)

-- Шаг 2: Пользователь долго редактирует профиль...
-- Шаг 3: Попытка сохранить изменения
UPDATE user_profiles
SET username = 'Alice_new', email = 'alice_new@mail.com', version = 6
WHERE user_id = 1 AND version = 5;

-- Шаг 4: Проверяем результат
-- Если ROW_COUNT = 1 -> успех, ROW_COUNT = 0 -> конфликт, нужно начинать заново.
```

**Плюсы**

Не держит долгих блокировок, отличная производительность и параллелизм, нет риска deadlock'ов.

**Минусы**

Риск ошибки при конфликте (пользователю придется переделывать работу).

Не подходит для высококонкурентных операций (тот же счетчик баланса), так как приведет к огромному числу повторных попыток (retry storms) и деградации производительности.

## Summary optimistic vs pessimistic locking

| Характеристика | Пессимистичная блокировка | Оптимистичная блокировка |
|---|---|---|
| Принцип | Конфликт вероятен, данные надо "застолбить" заранее. | Конфликт маловероятен, проверим в момент сохранения. |
| Механизм | SELECT ... FOR UPDATE (блокировки БД). | Поле version в таблице (логика приложения). |
| Когда применять | Высокая конкуренция за одни и те же данные (финансы, билеты, инвентарь). Пользователь не держит блокировку долго. | Низкая конкуренция, долгие операции редактирования (профиль пользователя, черновик статьи). |
| Параллелизм | Низкий. Операции встают в очередь. | Высокий. Операции выполняются параллельно. |
| Обработка конфликта | Ожидание или мгновенный отказ (NOWAIT). | Повтор операции приложением. |

# Партицирование, шардирование и реплицирование

Когда таблица вырастает до десятков и сотен миллионов строк, стандартные операции начинают работать медленнее: индексы разбухают, `VACUUM` не успевает, `Seq Scan` сканирует гигабайты.

В PostgreSQL и распределённых системах есть три ключевых подхода к решению этой проблемы:

- **Партицирование** — разрезание одной большой таблицы внутри одного сервера.
- **Шардирование** — разрезание одной большой таблицы по разным серверам.
- **Реплицирование** — копирование всей базы или её частей на другие сервера.

Несмотря на внешнюю похожесть, это принципиально разные механизмы с разными целями и последствиями.

## Партицирование

**Партицирование** — это логическое разбиение одной большой таблицы на несколько физических кусков (партиций) с прозрачным доступом через родительскую таблицу.

Каждая партиция — это обычная таблица со своими индексами, статистикой и файлами на диске. Родительская таблица — лишь прокси для маршрутизации запросов.

При вставке данных PostgreSQL сам определяет, в какую партицию попадает строка, и пишет её напрямую в нужный физический кусок.

**Зачем нужно**

- Ускоряет запросы с условием по ключу партицирования (выбирается одна-две партиции вместо сканирования всей большой таблицы).
- Упрощает удаление старых данных: вместо медленного `DELETE` можно сделать быстрый `DROP PARTITION`.
- Улучшает работу `VACUUM`: каждая партиция обслуживается отдельно, не блокируя остальные.
- Упрощает жизнь планировщику: меньше страниц для оценки, точнее статистика, лучше планы запросов.

**Когда становится актуально**

Как грубый ориентир: когда таблица переваливает за 50–100 ГБ или за 100–200 миллионов строк. Но точный порог зависит от характера нагрузки и железа.

Если таблица 20 ГБ, и `VACUUM` и запросы справляются, скорее всего, можно не парить мозг с партицированием, а сначала оттюнить индексы, память и запросы.

## Отличие от шардирования

| Признак | Партицирование | Шардирование |
|---|---|---|
| Где данные | Один сервер | Разные сервера |
| Кто управляет | Сам PostgreSQL | Приложение / proxy / coordinator |
| Транзакции ACID | Полные | Нет кросс-шардных ACID |
| JOIN между кусками | Можно, без проблем | Больно, дорого, или невозможно |
| Сложность | Настраивается declaratively | Архитектурная, ручная |

Иногда партицирование и шардирование путают, но если запомнить:

- партицирование — на одном сервере,
- шардирование — на разных серверах,

то путаницы не будет.

## Типы партицирования в PostgreSQL

В PostgreSQL (начиная с 10 версии) есть декларативное партицирование: не нужно вручную писать триггеры и правила — достаточно описать структуру.

### По диапазону (RANGE)

Данные разбиваются по диапазонам значений. Самый частый тип.

```
CREATE TABLE orders (
    id          BIGSERIAL,
    created_at  TIMESTAMPTZ NOT NULL,
    user_id     BIGINT,
    amount      NUMERIC
) PARTITION BY RANGE (created_at);

-- Партиции на каждый месяц
CREATE TABLE orders_2025_01 PARTITION OF orders
    FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
```

```
CREATE TABLE orders_2025_02 PARTITION OF orders
    FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');
```

#### Когда использовать

- Данные равномерно накапливаются по времени (логи, заказы, события).
- Часто удаляются старые данные: `DROP PARTITION` старых месяцев — дешёвая операция.

### По списку (LIST)

Данные разбиваются по конкретным значениям, не по диапазонам.

```
CREATE TABLE payments (
    id          BIGSERIAL,
    status      TEXT,
    amount      NUMERIC
) PARTITION BY LIST (status);

CREATE TABLE payments_pending PARTITION OF payments
    FOR VALUES IN ('pending');

CREATE TABLE payments_completed PARTITION OF payments
    FOR VALUES IN ('completed');
```

#### Когда использовать

- Поле-классификатор с небольшим количеством значений (статус, тип, регион).

### По хэшу (HASH)

Данные разбиваются по хэшу от указанных полей.

```
CREATE TABLE sessions (
    id          UUID,
    user_id     BIGINT,
    data        JSONB
) PARTITION BY HASH (user_id);

-- Создаём 8 партиций
CREATE TABLE sessions_p0 PARTITION OF sessions
    FOR VALUES WITH (modulus 8, remainder 0);
CREATE TABLE sessions_p1 PARTITION OF sessions
    FOR VALUES WITH (modulus 8, remainder 1);
-- ...
```

#### Когда использовать

- Нет естественного ключа для диапазонов или списков.
- Нужно равномерно распределить данные по партициям.
- Типичный кейс: сессии, где нет даты создания как основного фильтра, и ищем по `user_id`.

## Когда НЕ нужно партицировать

Партицирование — не silver bullet. Есть сценарии, когда оно либо не даст выигрыша, либо сделает хуже.

- Таблица меньше 10–20 ГБ и запросы работают адекватно.
- Основной паттерн запросов — точечный поиск по первичному ключу (индекса достаточно, без партицирования будет даже чуть быстрее из-за меньшей глубины планирования).
- 95% запросов не попадают под ключ партицирования — планировщик всё равно будет обходить все партиции.
- Количество партиций переваливает за несколько тысяч: планирование запроса начинает тормозить.

**Партицирование оправдано, когда у вас есть:**

1. Большая таблица, которая продолжает расти.
2. Основная масса запросов содержит условие по ключу партицирования.
3. Периодически нужно массово удалять старые данные.
4. `VACUUM` на таблице уже не справляется.

## Partition pruning

Главный механизм, дающий ускорение при партицировании — **partition pruning** (отсечение партиций).

Планировщик на этапе разбора запроса определяет, какие партиции точно не могут содержать нужные строки, и исключает их из плана.

```
-- Партицирование по created_at, ищем за один день
SELECT * FROM orders
WHERE created_at >= '2025-02-15'
  AND created_at <  '2025-02-16';
```

Планировщик поймёт: все данные лежат только в `orders_2025_02`, остальные партиции трогать не нужно.

**Важно:** pruning работает эффективно, если в `WHERE` участвует именно ключ партицирования. Если фильтруем только по `user_id`, а партицирование по `created_at`, придётся обойти все

партиции — чуда не будет.

## Шардирование

**Шардирование** — это техника распределения данных одной логической таблицы по нескольким физическим серверам (шардам).

Сам PostgreSQL не предоставляет встроенного шардирования. Оно реализуется на уровне приложения, либо через внешние инструменты (Citus, pg_shardman, fdw-based).

**Принцип работы**

1. Выбирается **ключ шардирования**: поле или набор полей, по которым данные равномерно раскладываются (например, `user_id`, `tenant_id`).
2. Определяется **функция шардирования**: хэш, диапазон или список.
3. Приложение (или proxy-слой) по значению ключа определяет, на каком шарде живёт запись, и направляет запрос туда.

```
# Пример: хэш-шардирование
шард = hash(user_id) % количество_шардов
```

**Когда нужно шардирование**

- Таблица выросла настолько, что один сервер не тянет ни по диску, ни по IOPS, ни по CPU.
- Размер базы вышел за пределы вертикального масштабирования (самый жирный сервер уже не помогает).
- Нагрузка на запись выше, чем способен обработать один инстанс.

**Ключевая проблема — кросс-шардные запросы**

Если все запросы привязаны к одному ключу шардирования (например, «все заказы пользователя 123»), маршрутизация простая.

Проблемы начинаются, когда нужны запросы, которые не привязаны к одному ключу:

```
-- Агрегация по всем пользователям
SELECT status, COUNT(*)
FROM orders
GROUP BY status;
```

Такой запрос должен уйти на все шарды, потом результаты нужно собрать и смержить. Это сложно, дорого и убивает саму идею шардирования.

Поэтому шардирование подходит в первую очередь для:

- OLTP с изолированными данными по tenant'ам (SaaS с tenant_id).
- Высоконагруженных систем, где 95% запросов завязаны на один ключ шардирования.

Шардирование ради «сделать красиво» почти всегда приводит к боли без выигрыша.

# Реплицирование

**Реплицирование** — это копирование данных с одного сервера на другие.

В отличие от партицирования и шардирования, где каждый кусок уникален, при реплицировании одни и те же данные живут на нескольких серверах.

**Основные цели:**

- **Отказоустойчивость**: если основной сервер умирает, реплика может занять его место.
- **Чтение с реплик**: разгрузить мастер, отправляя `SELECT` -запросы на реплики.
- **Резервное копирование без нагрузки на мастер**: `pg_dump` с реплики.
- **Географическое распределение**: реплика ближе к пользователям в другом регионе.

## Типы репликации

#### Физическая репликация

Копирует изменения на уровне страниц WAL (Write-Ahead Log).

- Побитово идентичные копии кластера.
- Репликация всей базы, нельзя реплицировать отдельные таблицы.
- Асинхронная или синхронная.
- Надёжно, проверено годами.

**Встроена в PostgreSQL**, настраивается через `primary_conninfo`, `recovery.conf` (в ранних версиях) и слоты репликации.

#### Логическая репликация

Копирует изменения на уровне строк (INSERT/UPDATE/DELETE), декодируя WAL в логические изменения.

- Можно реплицировать отдельные таблицы, а не всю базу.
- Можно реплицировать между разными версиями PostgreSQL (для мажорных обновлений без даунтайма).
- Можно фильтровать строки через публикации/подписки.

- Не требует побитовой идентичности серверов.

**Настраивается через:**

```
-- На источнике
CREATE PUBLICATION my_pub FOR TABLE orders, users;

-- На подписчике
CREATE SUBSCRIPTION my_sub
CONNECTION 'host=... dbname=...'
PUBLICATION my_pub;
```

**Когда использовать логическую репликацию**

- Нужна репликация не всей базы, а отдельных таблиц.
- Нужен online-апгрейд мажорной версии.
- Нужна репликация в другую СУБД или в систему аналитики.

## Синхронная vs Асинхронная репликация

| Характеристика | Асинхронная | Синхронная |
|---|---|---|
| Коммит на мастере | Не ждёт подтверждения от реплики | Ждёт записи WAL на реплику |
| Задержка | Возможна (lag) | Гарантированно без потерь |
| Производительность | Высокая | Ниже (зависит от сети и дисковой реплики) |
| Потеря данных при падении | Возможна (то, что не успело уйти) | Нет |

В большинстве систем используют асинхронную репликацию + мониторинг лага. Синхронную — когда потеря даже одной транзакции критична (финансы, бухгалтерия).

## Что реплицирование НЕ решает

Важно понимать: репликация не масштабирует запись.

Все `INSERT/UPDATE/DELETE` всё равно проходят через мастер. Реплики помогают разгрузить чтение, но если вы упираетесь в пишущую нагрузку — нужно шардирование или другое железо, а не больше реплик.

## Шпаргалка: что когда использовать

| Проблема | Решение | Почему |
|---|---|---|
| Большая таблица, долгие Seq Scan | Партицирование | Partition pruning, селективное сканирование |
| Много старых данных, нужно удалять | Партицирование | `DROP PARTITION` вместо `DELETE` |
| VACUUM не справляется с большой таблицей | Партицирование | VACUUM работает с маленькими партициями |
| Сервер перегружен чтением | Реплицирование | Чтение с реплик |
| Нужна отказоустойчивость | Реплицирование | Автоматический failover |
| Сервер не вывозит запись, диск кончился | Шардирование | Разносим запись по разным машинам |
| Мультитенантное SaaS | Шардирование по tenant_id | Изоляция данных, равномерная нагрузка |
| Аналитика без нагрузки на боевую базу | Логическая репликация | Реплицируем нужные таблицы в отдельный инстанс |

# Курсорная пагинация

Курсорная пагинация - пагинация по последнему отданному объекту.

**Курсор** - это ключ, в котором содержатся значения полей последнего отданного объекта, закодированные в токен.

Поля курсора должны быть теми, по которым мы сортируем при отдаче.

Например, я хочу отдавать вакансии:

Если я отдаю вторую страницу по 20:

```
SELECT * FROM vacancy
ORDER BY created_at, id DESC
LIMIT 20 OFFSET 20;
```

Но PostgreSQL придется считать все 20 + 20 строк и отдать последние 20.

На больших страницах и дальних страницах это будет большой проблемой.

То же самое можно организовать через курсор:

То есть допустим у меня последний отданный объект на первой странице имеет `id=100`, `created_at='2020-01-01 00:00:00+03'`

Тогда можно отдать просто 20 объектов после него и не вычитывать все строки до него.

```
SELECT * FROM vacancy
WHERE (created_at, id) < ('2020-01-01 00:00:00+03', '100')
ORDER BY created_at, id DESC
LIMIT 20;
```

Если есть индекс по этим полям, то будет еще быстрее.

**Плюсы относительно `LIMIT/OFFSET`**

1. Лучше производительность на больших выборках
2. При правильном выборе sort key можно не волноваться о дедубликации или сдвиге данных (допустим добавили строку на первую таблицу, с LIMIT/OFFSET если мы запрашиваем вторую страницу, то эту запись мы уже потеряем)
3. Хорошо подходит для infinite scroll ленты

**Минусы относительно `LIMIT/OFFSET`**

1. Сложнее реализовать
2. Сложнее прыгать по страницам, потому что о номере страниц нигде нет информации
3. Нужен устойчивый порядок и тщательный выбор ключей, иначе будут дубликаты или потеря строк
4. Тяжелее возвращаться, потому что нужно инвертировать сортировку по ключам

**Тонкости**

Курсорная пагинация хорошо работает с индексом.

Индекс должен быть то тем же полям и в том же порядке, по которому сортируем выборку.

**Пример**

Допустим, у нас список вакансий:

```
SELECT id, title, created_at
FROM vacancies
WHERE company_id = $1
  AND (created_at, id) < ($2, $3)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

Индекс:

```
CREATE INDEX idx_vacancies_company_created_id
ON vacancies (company_id, created_at DESC, id DESC);
```

Тогда все будет хорошо:

- фильтрация по company_id
- курсор по (created_at, id)
- сортировка совпадает с индексом